

CSE 180 - Introduction to Robotics

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced

<https://sites.ucmerced.edu/scarpin>
<https://robotics.ucmerced.edu>



Administrative Details

Instructor Prof. Stefano Carpin

Office Hours: Tuesday 12:00pm-1:00pm (Room SE2-222). Other times by appointment only (to be arranged via email.)

Teaching Assistants

- Marcos Zuzuarregui
- Hamid Rajabi

TAs will communicate their office hours on Canvas



Canvas/CatCourses

- Canvas/CatCourses will be used for all aspects of this class (announcements, slides, assignments, grades, etc.). It is expected that you **visit the course site regularly**.



Course Materials

- Textbook freely available online: <https://robotics.ucmerced.edu/MRTP>
- Code examples: <https://github.com/stefanocarpin/MRTP>



Course Grading

- Homeworks: 10%
- Labs: 20%
- Final Project: 20%
- Midterm (closed book, October 16, 2025 – date to be confirmed): 20%
- Final (closed book, December 13, 2025) : 30%



Policies

See also syllabus for more details.

Disability Accomodations : <https://access.ucmerced.edu/>

Academic Honesty : UCM Academic Honesty Policy

Supplemental Computer Science Dept Policy : link

Communications : Email communications for this course **must** be sent using CatCourses' builtin messaging tool. Emails directly sent to me are likely to receive a belated response or being ignored. Emails not sent from @ucmerced.edu account will not be answered.

Grading Questions : if you have questions about grades, please **contact your TA first within 7 days from when the grade is communicated**. If you still have questions, concerns, then you can contact me (forwarding all previous email exchanges.)

Classroom etiquette : see expectations posted on CatCourses



Homeworks

- due 7 days after they are assigned.
- **must** be submitted through catcourses (no exceptions – submissions via email will be rejected).
- **grace period**: submissions received 72h after the posted deadline will be subject to a 50% penalty; submissions received more than 72h after the deadline will not be graded.
- solutions **must** be individual. You are encouraged to discuss with others, but when you start writing the solution it must be yours and yours only.
- **ChatGPT/AI etc**: they can surely solve your assignments, but you will not be allowed to use it during the exams (nor during a job interview ...).



- you must attend the session have signed up for.
- TAs will assist with ROS specific questions, but will not solve the exercises for you.
- the first week of lab you will setup ROS 2 on your machine or the lab machine; this will be a graded assignment.
- you can use generative AI to solve the assignments; however, the TAs will not help with debugging code generated by LLMs or other AI tools.



Suggestions on how to fail this class

- Skip classes.
- Start studying late.
- Come to class but pay attention to your phone rather than what is being discussed.
- Study only when an assignment/exam approaches.
- Copy your homeworks from others or ChatGPT/Gemini/whatever without trying to understand the solution.
- Count on copying during the exams.



Introduction

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced
<https://sites.ucmerced.edu/scarpin>
<https://robotics.ucmerced.edu>



Robot

- From the Czech word *robota* meaning *forced labor*.
- first appeared in 1920 in the play *R.U.R.* '*Rossum's Universal Robots*' by writer Karl Čapek



Source: <http://www.umich.edu/ engb415/literature/pontee/RUR/RURsmry.html>



Autonomy / Autonomous Robot

Autonomous: from the Greek words *auto* (self) and *nomos* (law). Today is interpreted as self governed.



Definitions of Robot

Wikipedia a machine – especially one programmable by a computer – capable of carrying out a complex series of actions automatically
(<https://en.wikipedia.org/wiki/Robot>).

Merriam-Webster a machine that resembles a living creature in being capable of moving independently (as by walking or rolling on wheels) and performing complex actions (such as grasping and moving objects)
(<https://www.merriam-webster.com/dictionary/robot>)

RIA A robot is defined as a reprogrammable, multifunctional manipulator designed to move material, parts, tools, or specialized devices through various programmed motions for the performance of a variety of tasks (RIA, 1979).



Video Interlude...

Robot Taxonomies

Mobility

- aerial
- wheeled
- legged
- marine

Applications

- industrial
- service
- medical
- field
- social



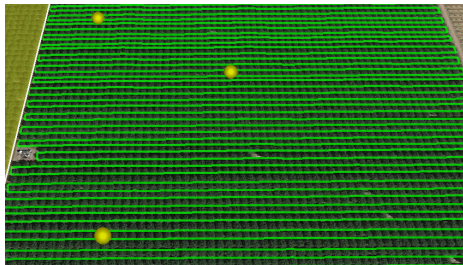
Mobile Robots

- a robot not constrained to remain in a preassigned area
- not a formal or exhaustive definition
- may operate in a structured or unstructured environment



Functionalities

- Planning
- Localization
- Navigation and Obstacle Avoidance



Robots Feature Multiple Interacting Subsystems



Open loop operation

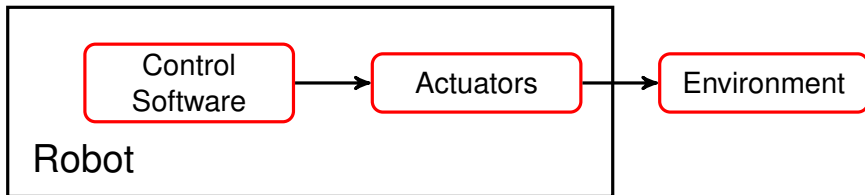


Figure: A robot operating in open loop.



Robot Modeling: Agent View

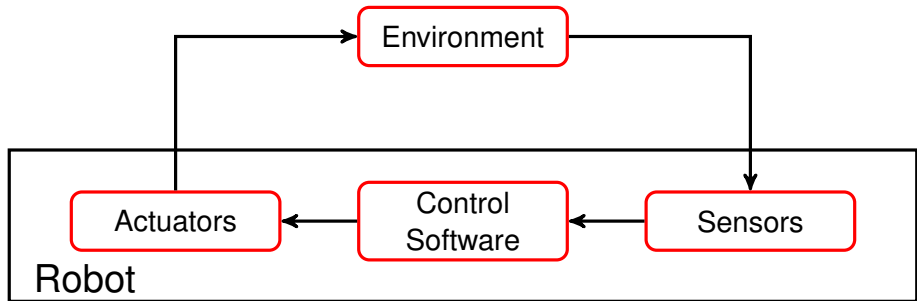


Figure: An agent view of a mobile robot.



Robot Modeling: Dynamical System View

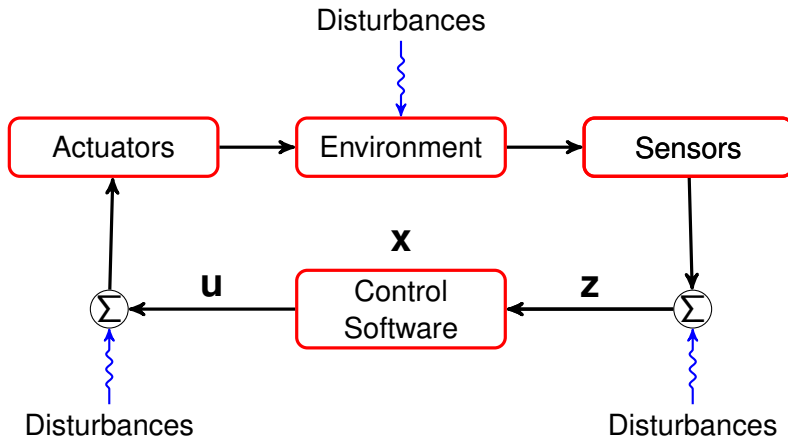
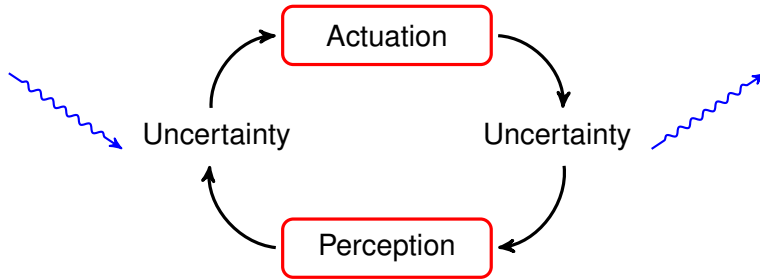


Figure: A dynamical system view of a mobile robot.



Uncertainty Cycle



Dynamical Systems Model

Dynamical system : a system whose behavior changes over time

Feedback : coupling between two systems influencing each other

State of a robot: position, velocity, joint values, ...

- x** state of the system
- u** input give to the system (actuators)
- z** information extracted from the system (sensors)



Dynamical system equations

Continuous time model:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, t) \quad (1)$$

$$\mathbf{z} = h(\mathbf{x}, \mathbf{u}, t) \quad (2)$$

Eq. (1) is the **state transition equation** and Eq. (2) is the **state observation equation**

Discrete time model:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t, t) \quad (3)$$

$$\mathbf{z}_t = h(\mathbf{x}_t, \mathbf{u}_t, t). \quad (4)$$



Time Invariant Systems

Continuous equations

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) \quad (5)$$

$$\mathbf{z} = h(\mathbf{x}, \mathbf{u}) \quad (6)$$

Discrete equations

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t) \quad (7)$$

$$\mathbf{z}_t = h(\mathbf{x}_t, \mathbf{u}_t) \quad (8)$$

Assumption of sensor reading independent from the input

$$\mathbf{z}_t = h(\mathbf{x}_t) \quad (9)$$



State evolution

$$\mathbf{x}_0 \xrightarrow{\mathbf{u}_1} \mathbf{x}_1 = f(\mathbf{x}_0, \mathbf{u}_1) \xrightarrow{\mathbf{u}_2} \mathbf{x}_2 = f(\mathbf{x}_1, \mathbf{u}_2) \xrightarrow{\mathbf{u}_3} \mathbf{x}_3 = f(\mathbf{x}_2, \mathbf{u}_3) \rightarrow \dots \rightarrow \mathbf{x}_n = f(\mathbf{x}_{n-1}, \mathbf{u}_n)$$



State and sensor acquisition

$$\begin{array}{ccccccc} \mathbf{x}_0 & \xrightarrow{\mathbf{u}_1} & \mathbf{x}_1 = f(\mathbf{x}_0, \mathbf{u}_1) & \xrightarrow{\mathbf{u}_2} & \mathbf{x}_2 = f(\mathbf{x}_1, \mathbf{u}_2) & \xrightarrow{\mathbf{u}_3} & \mathbf{x}_3 = f(\mathbf{x}_2, \mathbf{u}_3) \longrightarrow \dots \\ & & \downarrow & & \downarrow & & \downarrow \\ & & h(\mathbf{x}_1) & & h(\mathbf{x}_2) & & h(\mathbf{x}_3) \end{array}$$

$$\begin{array}{ccccccc} \mathbf{x}_0 & \xrightarrow{\mathbf{u}_1} & \mathbf{x}_1 = f(\mathbf{x}_0, \mathbf{u}_1) & \xrightarrow{\mathbf{u}_2} & \mathbf{x}_2 = f(\mathbf{x}_1, \mathbf{u}_2) & \xrightarrow{\mathbf{u}_3} & \mathbf{x}_3 = f(\mathbf{x}_2, \mathbf{u}_3) \longrightarrow \dots \\ & \searrow & \downarrow & \searrow & \downarrow & \searrow & \downarrow \\ & \mathbf{z}_1 = h(\mathbf{x}_1, \mathbf{u}_1) & & \mathbf{z}_2 = h(\mathbf{x}_2, \mathbf{u}_2) & & \mathbf{z}_3 = h(\mathbf{x}_3, \mathbf{u}_3) \end{array}$$



Robot (software) Controller

- our eventual high level objective would be to get a function π that defines the action to take in each state so that the robot reaches a desired state (planning view)

$$\pi(\mathbf{x}_t) = \mathbf{u}_{t+1}$$

- the state, however, is almost invariably not known with certainty. So we also need an algorithm to determine the state (estimation view)

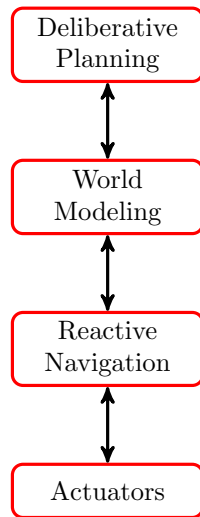
$$\mathbf{x}_t = g(\mathbf{x}_1, \dots, \mathbf{x}_{t-1}, \mathbf{z}_1, \dots, \mathbf{z}_{t-1})$$

- these functions are computed by the algorithms/programs we will study in this course
- must be pursued in parallel



Robot Software Architecture

- principle for modular decomposition of a complex software system
- multiple approaches and solutions
- ROS based architectures emphasize data exchange and asynchronous interactions between modules



ROS 2 – Robot Operating System

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced

<https://sites.ucmerced.edu/scarpin>

<https://robotics.ucmerced.edu>



What is ROS?

ROS – Robot Operating System is *not* an operating system but but a framework to develop robot control software.

- 1 a software middleware enabling secure communications between components using different patterns (e.g., asynchronous, synchronous, and more);
- 2 a collection of tools to simplify the development and debugging of complex robotic applications;
- 3 implementations of numerous algorithms solving basic robotic problems that can be composed together to develop more complex functionalities (e.g., localization, navigation, planning, etc.);
- 4 definitions of various data types (messages) to process and exchange data commonly needed to implement robotic applications (e.g., quaternions, transformation matrices, sensor data, etc.).



ROS Distributions

Distribution	Supported Platforms
Humble	Ubuntu 22.04, Windows 10
Iron	Ubuntu 22.04, Windows 10
Jazzy	Ubuntu 24.04, Windows 10
Kilted	Ubuntu 24.04, Windows 10

Table: Recent ROS 2 distributions.

In this course we will use **Jazzy**



How Can I practice with ROS?

- install Ubuntu on a dedicated partition [OR]
- install Ubuntu through Docker (works for MacOS, Windows, or Linux distributions different from Ubuntu 22)
- follow instructions on <https://github.com/stefanocarpin/MRTP/>
- One more alternative: create a free account on <https://www.theconstructsim.com/> and use a fully online solution
- ROS2 can also run on Windows via WSL



Some Important ROS concepts

- node
- topic
- message
- service and action
- package and workspace



Nodes

A *ROS program* consists of various interacting computational threads called **nodes**:

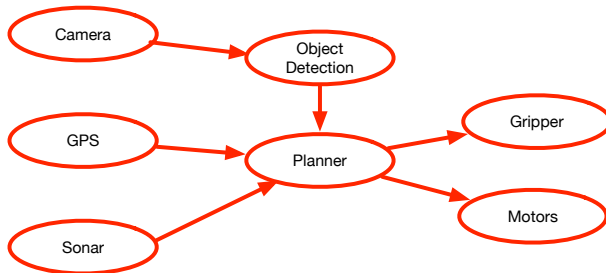
- each node is independently executed
- nodes are executed *concurrently*
- nodes can be written using various ROS client libraries available in different languages (C++, Python, Java, Lisp, ...)
- nodes interact with each other through messages, services, actions
- node decomposition facilitates code reuse and debugging, and increases robustness
- nodes are *loosely coupled*
- typical ROS applications mix ROS provided nodes with user-developed nodes



Multiple Interacting Subsystems



Multiple Interacting Subsystems – Possible ROS decomposition



ROS Nodes

- a ROS application typically runs both nodes written by the programmer to solve a specific task, as well as nodes part of the ROS standard distribution
- for many basic/common tasks, ROS provides implementations through nodes ready to use
- learning how to navigate the ROS documentation to take advantage of available code is essential to develop robust applications and reduce development time



Topics

A **topic** is an asynchronous, unidirectional stream of messages of a defined type.

asynchronous : publisher and subscriber do not rely on a shared time line

unidirectional : messages always flow from the publisher(s) to the subscriber(s)

stream : messages are sent sequentially and the relative order is not swapped

- nodes *subscribe* (receive from) or *advertise/publish* (send to) topics

Important features:

- Communication through topics is *many-to-many*
- All messages exchanged through a topic have the same type
 - also (improperly) called the type of the topic

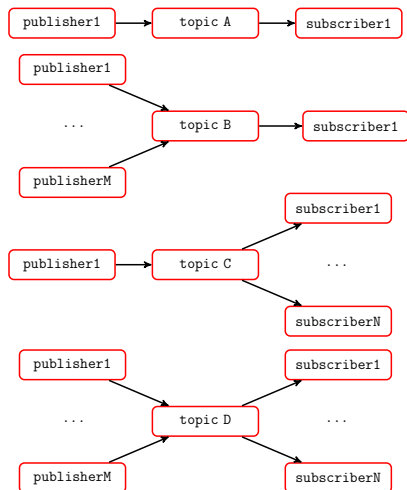


Topics

- handshake happens via the topic name (a string)
- nodes can subscribe/publish to many topics at once
- multiple nodes can use the same topic
- messages through a topic are queued using a finite length buffer (FIFO)



Topics are many-to-many



Packages and workspaces

package : a container of ROS entities (nodes, launch files, datasets, ...)

- (almost) **everything** in ROS is located inside a package.
- the fully qualified name of an entity is obtained combining the name of the entity with the name of the package (similar to namespaces in C++)
- packages are stored in folders and each package includes various subfolders

workspace : a container of packages

To automate and expedite the recompilation process, packages and workspaces must be structured following a well defined layout (see **colcon** commands later on)

- **Why packages?** Keep things organized and avoid name clashes



Overlay and Underlay

underlay is the workspace including all packages shipped with the ROS 2 distribution

overlay is any additional workspace including the package you develop or get from third party

- underlay and overlay must be made *visible* before they can be used (sourced)

- *sourcing the setup* makes the underlay visible

```
source /opt/ros/jazzy/setup.bash
```

This command should be added to the startup script

- overlays are made visible with a different command



The command line tool `ros2`

- `ros2`: entry point for numerous fundamental operations nodes, topics, packages and more
- syntax:
`ros2 <command> <positional parameters>`
- example: list all packages available
`ros2 pkg list`
- to list all commands:
`ros2 -h`
- to get help about a specific command
`ros2 <command> -h`



ros2 run

- `ros2 run` is used to start ROS executables (user never directly starts nodes)
- needs (at least) name of the package and of the executable
`ros2 run <packagename> <executablename>`
- accepts additional parameters; will be discussed later



Running nodes

- simplest possible example: run two nodes that exchange information through a topic (sender/receiver)
- nodes are part of the `demo_nodes_cpp` package
- `ros2 run demo_nodes_cpp talker`
- `ros2 run demo_nodes_cpp listener`

Try this example and try restarting the nodes at different times or starting multiple instances of the nodes.



Running Turtlesim

Turtlesim is a simple application designed to familiarize and experiment with various ROS concepts

```
ros2 pkg executables turtlesim
```

List of executables

```
turtlesim draw_square
```

```
turtlesim mimic
```

```
turtlesim turtle_teleop_key
```

```
turtlesim turtlesim_node
```

Let's teleoperate the turtle running the nodes `turtlesim_node` and `turtle_teleop_key`:

```
ros2 run turtlesim turtlesim_node
```

```
ros2 run turtlesim turtle_teleop_key
```



Command Line Debugging Tools

- `ros2 node`
- `ros2 topic`
- `rqt_graph`
- `ros2 interface`



ros2 node

- `ros2 node list`: **show all the running nodes**
- `ros2 node info [nodename]`: **shows detailed information about a running node**
 - additional options available; see documentation



ros2 topic

- command line tool providing information about currently active topics
- typically used together with `ros2 interface` (see next slides)
- To get list of all topics
`ros2 topic list`
- typical output (e.g., when running `turtle_sim`)

```
/parameter_events  
/rosout  
/turtle1/cmd_vel  
/turtle1/color_sensor  
/turtle1/pose
```



ros2 topic

- To get information about the type of message exchanged through a topic:

```
ros2 topic type /turtle1/cmd_vel
```

- typical output

```
geometry_msgs/msg/Twist
```

- to get more info about a topic:

```
ros2 topic info /turtle1/cmd_vel
```

```
Type: geometry_msgs/msg/Twist
```

```
Publisher count: 1
```

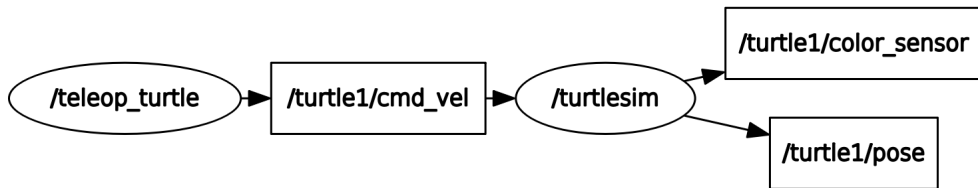
```
Subscription count: 1
```

- to see the messages transmitted through a topic: `ro2 topic echo [topicname]`

- type `ros2 topic -h` to see for full documentation

The ROS graph

- Useful debugging tool to make sure nodes are properly communicating
- run it with `rqt_graph`
- GUI enables/disables various components to be visualized



Messages

- a **message** is an instance of a data structure exchanged between nodes
- messages are typed
- messages can contain basic data types or other messages; very similar to a C++ `struct`
- a **topic** is the communication channel through which **messages** are exchanged



ros2 interface

- command line utility to get information about the structure of messages.
- **typical use:** `ros2 interface show [messagename]`
- **e.g.** `ros2 interface show geometry_msgs/msg/Twist` produces the output:

```
Vector3  linear
  float64 x
  float64 y
  float64 z
Vector3  angular
  float64 x
  float64 y
  float64 z
```

- we can also use `ros2 interface show geometry_msgs/msg/Vector3` to determine the structure of `Vector3`, and so on
- type `ros2 interface -h` to see for full documentation



using `ros2 topic` to publish messages

- useful to test subscribers
- uses YAML language to map strings into messages (give the command on just one line)

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist  
'{linear: {x: 0.1, y: 0.0, z: 0.0},  
angular: {x: 0.0,y: 0.0,z: 0.0}}'
```

- see online documentation for full details



List `ros2` commands (partial list)

Command	Scope
<code>action</code>	Command related to actions
<code>bag</code>	Command related to recording and replaying data
<code>component</code>	Command related to components
<code>doctor</code>	Diagnostic functions
<code>interface</code>	Information about interfaces (aka messages)
<code>launch</code>	Runs a launch file
<code>lifecycle</code>	Information about nodes lifecycle
<code>node</code>	Retrieves information about running nodes
<code>param</code>	Set/get parameters
<code>pkg</code>	Information about packages and package creation
<code>run</code>	Runs an executable from a package
<code>service</code>	Command related to services
<code>topic</code>	Information and interaction with topics

Services and Actions

service : remote procedure call between nodes.

- suited for quick computations, e.g., turn on or reconfigure a sensor
- returns a result

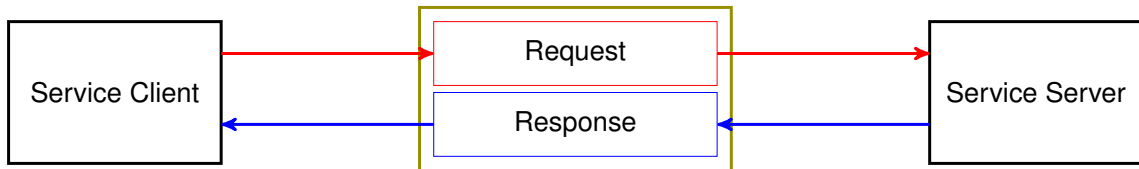
action : *non-blocking* remote procedure call between nodes

- suited for complex tasks, e.g., navigate to a far away location
- returns a result at the end, and updates while executing

Services and actions use messages to pass information between involved nodes



Services



Interacting with services

- `ros2 service list` shows all available services
- each service has a type, i.e., a is implemented as an exchange of a message of a given type. To determine the type of a service:
`ros2 service list -t`
- the message associated with a service has one section for the request and one section for the response
- `ros2 interface` can be used to determine the structure of a service request/response



Interacting with services

- `ros2 interface show turtlesim/srv/Spawn`

```
float32 x
```

```
float32 y
```

```
float32 theta
```

```
string name #Optional. A unique name will be created and returned  
if this is empty
```

```
---
```

```
string name
```



Interacting with services

- `ros2 service` can be used to initiate service calls and receive responses

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 1, y: 1,  
  theta: 0,  name:  'T1'}"
```

- output

```
requester: making request: turtlesim.srv.Spawn_Request(x=1.0,  
y=1.0,  theta=0.0,  name='T1')
```

```
response:
```

```
turtlesim.srv.Spawn_Response(name='T1')
```

Try `ros2 topic list` for the list of topics available now...



Actions

- suited for tasks taking longer to execute (e.g., navigate to a point)
- provide back to caller both *response* at the end, and *feedback* during execution
- can be interrupted by caller before they finish
- differently from services, actions are non-blocking



Actions



Interacting with actions

- `ros2 action list`: lists all available actions
- `ros2 action list -t`: lists all available actions and their type
`/turtle1/rotate_absolute [turtlesim/action/RotateAbsolute]`
- `ros2 interface show turtlesim/action/RotateAbsolute`
- **output:**
The desired heading in radians
`float32 theta`

The angular displacement in radians to the starting position
`float32 delta`

The remaining rotation in radians
`float32 remaining`



Interacting with actions

- `ros2 action send_goal`: can be used to initiate an action
`ros2 action send_goal /turtle1/rotate_absolute`
`turtlesim/action/RotateAbsolute '{theta: 0.5}'`



ROS Launch Files

- XML files to specify nodes to be run and their parameters (can also be written in other languages)
- reside in a package (in a folder called `launch`), and have extension `.launch.xml`
- can be run in two ways:
 - 1 `ros2 launch <packagename> <launchfilename>`
 - 2 `ros2 launch <absolutefilepath>`
- use the second form for debugging only; everything should go into a package



ROS Launch File Example

```
<launch>
  <node pkg="demo_nodes_cpp" exec="talker" name="talkerA" />
  <node pkg="demo_nodes_cpp" exec="talker" name="talkerB" />
  <node pkg="demo_nodes_cpp" exec="listener" />
</launch>
```

On GitHub: [MRTP/MRTP/unsorted/topics.launch.xml](https://github.com/MRTP/MRTP/unsorted/topics.launch.xml)



ROS Launch File Example with Separate Terminals

```
<launch>
  <node pkg="turtlesim" exec="turtlesim_node" name="turtlesim" />
  <node pkg="turtlesim" exec="turtle_teleop_key" name="teleop_key"
    launch-prefix="gnome-terminal_--" />
</launch>
```

On GitHub: [MRTP/MRTP/unsorted/turtle.launch.xml](https://github.com/MRTP/MRTP/unsorted/turtle.launch.xml)



ROS Programming – Basics

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced

<https://sites.ucmerced.edu/scarpin>

<https://robotics.ucmerced.edu>



Build tool and build system

- a ROS application consists of multiple nodes, possibly written in different languages and belonging to different packages
- **build system**: `ament_cmake` – an extension of CMake. Builds a single package
- **build tool**: `colcon` – uses the build system to build multiple packages, analyzes dependencies, etc.
- in ROS we use both



Creating and building a workspace

Before creating your packages, you should create your own workspace which will include your packages

```
$ mkdir -p CSE180/src
$ cd CSE180
$ colcon build
```

- you just built an empty workspace
- `colcon build` **must** be executed from the workspace folder
- packages go in the `src` folder
- `colcon` created various folders and files. To make your workspace visible to ROS as an *overlay* you **must** execute the following command from the workspace folder

```
$ . install/local_setup.bash
```
- official documentation suggests to do this from a shell different from where you run `colcon build`

Creating a package

- Must use `ros2 pkg` to create folders with the right structure

```
$ cd CSE180/src
```

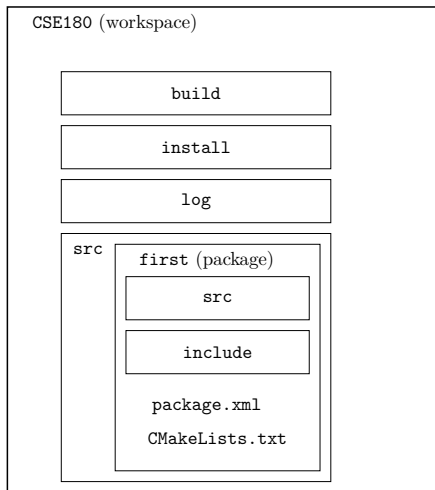
```
$ ros2 pkg create first
```

- just created a package called `first`
- packages **always** go into the `src` folder of the workspace
- besides folders, created files `CMakeLists.txt` and `package.xml`
- erasing a package? just remove its folder
- can also indicate the build system

```
ros2 pkg create --build-type ament_cmake first
```
- the package can now be built with `colcon`



Structure of a workspace and packages



package.xml

- an XML file included in every package
- includes information useful to build and distribute packages (e.g., dependencies)
- basic skeleton created by `ros2 pkg create`
- you typically have to update the list of package dependencies



package.xml example

MRTP/MRTP/unordered/package.xml



CMakeLists.txt

- CMake file
- specifies sources, libraries, dependencies, etc.
- basic skeleton created by `ros2 pkg create`; most can be used “as is”
- typically must add some entries for each executable in the package (source, packages it depends on, and more)
- more later...



Creating ROS executables

- built on top of `rcl`: ROS client library
- `rclcpp` exposes `rcl` through a set of C++ classes and functions
- `rclpy` offers equivalent functionalities for Python
- `rclcpp` is part of a package called `rclcpp`; all code we will write depends on this package



ROS executable typical structure

- 1 initialize ROS system
 - 2 instantiate a node, and setup communication via topics (services, actions)
 - 3 *do the job* – typically exchanging messages, calling/offering services, etc.
 - 4 shut down
- **spinning**: the operation of taking care of incoming and outgoing messages through topics; almost all nodes must call one of the spinning functions



First ROS Node: publishing

Create a new package called `talklisten`

```
cd ~  
mkdir -p MRTP/src  
cd MRTP/src  
ros2 pkg create --build-type ament_cmake talklisten
```

Create file

```
MRTP/MRTP/src/talklisten/src/talker.cpp
```



Take Home Messages

- always include `rclcpp.hpp` and create a node handler (pointer)
- familiarize yourself with the package `example_interfaces` (more later)
 - use `ros2 interface show` to find out the structure of its messages
- use fully qualified names for ROS classes (not a strict requirement, just common use)
- to keep things simple, in the beginning we will have just one node per executable
 - **composition** is an approach through which an executable generates multiple nodes (more later...)



Second ROS Node: subscribing

```
MRTP/MRTP/src/talklisten/src/listener.cpp
```



Take Home Messages

- subscribers must call `ros::spin` or `ros::spin_some` (see later) to retrieve messages
- messages are processed in *callback* functions
- use `RCLCPP_INFO` and associated macros to print to the screen
- callback functions are supposed to be quick to terminate because they may be called very frequently



spin VS spin_some

- `spin_some`: when called all messages pending are processed (sent or received); then the function returns
- `spin`: non returning function that keeps checking for messages
- think carefully about the logic of your node before deciding which one to call



Updating package.xml

```
MRTP/MRTP/src/talklisten/package.xml
```



Updating CMakeLists.txt

```
MRTP/MRTP/src/talklisten/CMakeLists.txt
```



Build and run

- move to the workspace folder
- run `colcon build`
- source the overlay (different shell)
- use `ros2 run` to run the executables



More ROS examples

- `M RTP / M RTP / src / multipletopics / src / multipublish . cpp`
- `M RTP / M RTP / src / multipletopics / src / multisub . cpp`
- **Let's add our own launch file with multiple consoles**
`M RTP / M RTP / src / multipletopics / launch / multiplet1 . launch . xml`
- **update** `package . xml`
`M RTP / M RTP / src / multipletopics / package . xml`
- **and** `CMakeLists . txt`
`R TP / M RTP / src / multipletopics / CMakeLists . txt`



ROS STREAMS

RCLCPP_INFO : information to the user

RCLCPP_DEBUG : information for debugging

RCLCPP_WARN : communicate unusual conditions (warnings).

RCLCPP_ERROR : diagnostic about recoverable errors.

RCLCPP_FATAL : diagnostic about unrecoverable errors.



Standard messages

C++ data type	ROS message
<code>bool</code>	<code>example_interfaces::msg::Bool</code>
<code>char</code>	<code>example_interfaces::msg::Char</code>
<code>unsigned char</code>	<code>example_interfaces::msg::UInt8</code>
<code>int</code>	<code>example_interfaces::msg::Int32</code>
<code>unsigned int</code>	<code>example_interfaces::msg::UInt32</code>
<code>long int</code>	<code>example_interfaces::msg::Int64</code>
<code>unsigned long int</code>	<code>example_interfaces::msg::UInt64</code>
<code>float</code>	<code>example_interfaces::msg::Float32</code>
<code>double</code>	<code>example_interfaces::msg::Float64</code>

Table: Correspondences between C++ elementary data types and ROS standard messages.

- `std::string` can be sent using messages of type `example_interfaces::msg::string`



Sending/Receiving Multidimensional Arrays

- For arrays or multidimensional structures, we use `XXXMultiArray`, e.g.,
`Int32MultiArray`
- **let us use** `ros2` interface **to understand the structure of** `Int32MultiArray`
- `M RTP/M RTP/src/examples/src/sendarray.cpp`
- `M RTP/M RTP/src/examples/src/subarray.cpp`
- `M RTP/M RTP/src/examples/src/sendmatrix.cpp`
- `M RTP/M RTP/src/examples/src/submatrix.cpp`



MultiArray and MultiArrayLayout

- data is stored linearly in `data`; `layout` maps multidimensional indices into a linear structure
- `stride` is used to store separation between successive elements
- store outer dimensions first
- If the multidimensional structure has n dimensions $A_1 \times A_2 \times A_3 \times \dots \times A_n$, then the stride S_i of the i -th dimension is $S_i = A_i \times A_{i+1} \times \dots \times A_n$. An element is identified by n indices $(i_1, i_2, \dots, i_n)$. Its position in the linear data structure is (assuming 0 offset):

$$i_1 \cdot S_2 + i_2 \cdot S_3 + \dots + i_{n-1} \cdot S_n + i_n$$



Subscribing and Publishing from the Same Node

- often nodes *both* publish and subscribe to topics
- two approaches:
 - publish from the handler function
 - use `spin_some`
- `M RTP / M RTP / s r c / e x a m p l e s / s r c / p u b s u b . c p p`
- publisher objects shall not be declared in the handler function



Kinematics

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced
<https://sites.ucmerced.edu/scarpin>
<https://robotics.ucmerced.edu>



Motivation and Objectives

Kinematics: the study of motion without considering its causes (position, velocity, acceleration).

Motivation

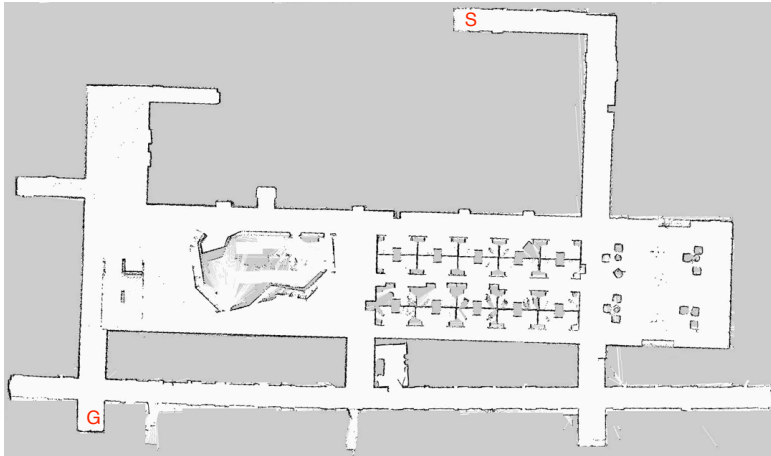
- robots operate in the three dimensional world
- algorithms must reason and make decisions about the space in which robots move

Objective

- introduce mathematical models to represent objects in space
- develop efficient algorithms to manipulate these models
- illustrate how these concepts are implemented in ROS



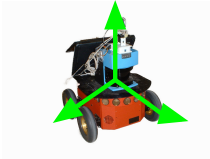
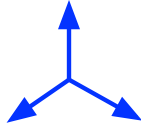
Why Kinematics?



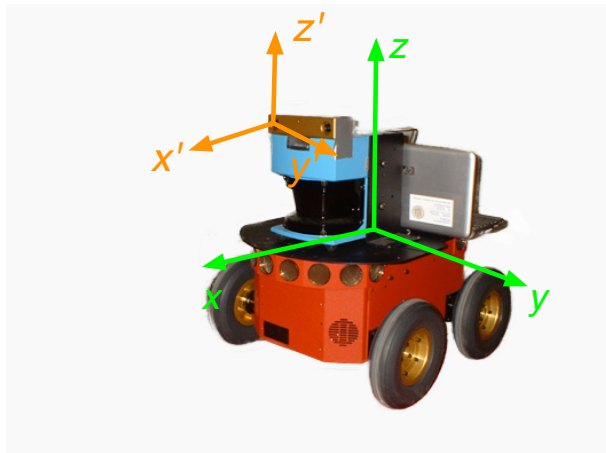
Why Kinematics?



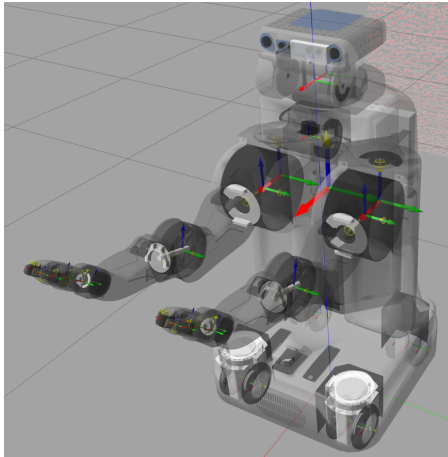
Why Kinematics?



Why Kinematics?



Why Kinematics?



Terminology and Assumptions

- we assume the existence of a so called *world* frame, i.e., a reference frame with respect to which coordinates and angles are expressed
- we abstract the robot as a *rigid object*
- its pose in the world is defined by its *position* and *orientation*
- position and orientation will be collectively called **pose**
- assumption: a rigid frame is attached to the robot
 - *Rigidly attached* means that whenever the robot moves the frame moves, and vice versa
- we reason about the pose of the frame as a proxy for the pose of the robot



Background and Notation

Definition

A body $\mathcal{B}$ is said to be a *rigid body* if the Euclidean distance between any two points in $\mathcal{B}$ is constant.

Definition

A frame of reference is given by an origin point O and three orthonormal vectors $\mathbf{x}$, $\mathbf{y}$ and $\mathbf{z}$ called axes satisfying the so-called right hand rule, i.e.,

$$\mathbf{x} \times \mathbf{y} = \mathbf{z} \quad \mathbf{y} \times \mathbf{z} = \mathbf{x} \quad \mathbf{z} \times \mathbf{x} = \mathbf{y}.$$

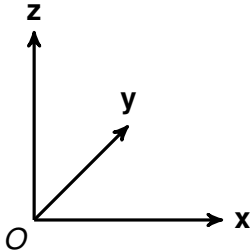
Orthonormal definition:

$$\mathbf{x} \cdot \mathbf{x} = 1 \quad \mathbf{y} \cdot \mathbf{y} = 1 \quad \mathbf{z} \cdot \mathbf{z} = 1 \quad \mathbf{x} \cdot \mathbf{y} = 0 \quad \mathbf{y} \cdot \mathbf{z} = 0 \quad \mathbf{z} \cdot \mathbf{x} = 0.$$

We indicate a frame as $O - \mathbf{xyz}$ and give symbolic names to frames, like A , B , etc.



Frame Representation



Note the orientation of the axes (right handed)

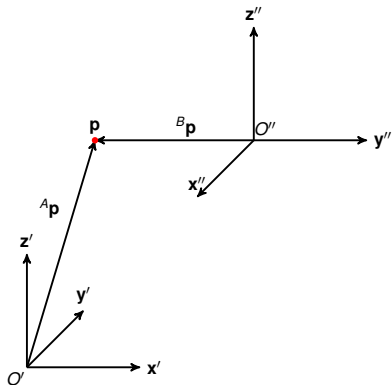


Coordinates in Multiple Frames

- a point $\mathbf{p}$ in $\mathbb{R}^n$ is represented by n values (coordinates)
- coordinate values are **always** expressed in a reference frame
- in robotics application there will be many reference systems; so it is important to specify to which reference system the coordinates are referred to
- a leading superscript is used to indicate the reference system
- ${}^A\mathbf{p}$ indicates the vector with the coordinates of point $\mathbf{p}$ referred to reference frame A while ${}^B\mathbf{p}$ indicates the coordinates of the same point but referred to reference system B



Coordinates in Multiple Frames

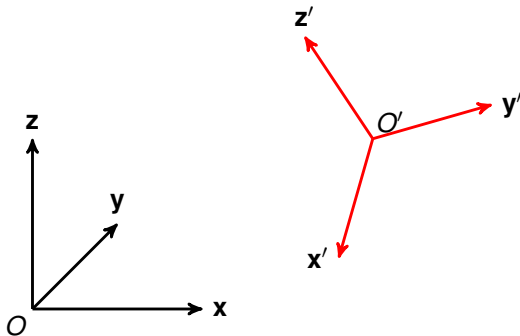


In this case ${}^A\mathbf{p} = [0.6 \ 2 \ 2]^T$ and ${}^B\mathbf{p} = [0 \ -1.4 \ 0]^T$



Representing a Frame with Respect to a Different Frame

Often we need to represent a frame $B = O' - \mathbf{x}'\mathbf{y}'\mathbf{z}'$ with respect to another frame $A = O - \mathbf{x}\mathbf{y}\mathbf{z}$.

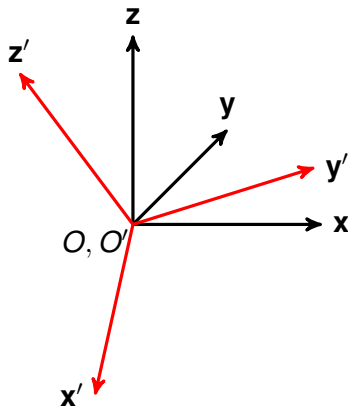


Representing a Frame with Respect to a Different Frame

- must represent its origin O' , i.e., a point. We know how to do that, i.e., ${}^A\mathbf{p}$ or ${}^AO'$.
- must represent its three orthonormal axes $\mathbf{x}'\mathbf{y}'\mathbf{z}'$
- consider translating frame B so that its origin coincides with frame A . Then take the coordinates of the endpoints of $\mathbf{x}'\mathbf{y}'\mathbf{z}'$



Representing a Frame with Respect to a Different Frame



Representing a Frame with Respect to a Different Frame

Coordinates of the end-point of axis $\mathbf{x}'$

$$A_{\mathbf{x}'} = \begin{bmatrix} \mathbf{x}' \cdot \mathbf{x} \\ \mathbf{x}' \cdot \mathbf{y} \\ \mathbf{x}' \cdot \mathbf{z} \end{bmatrix}$$

Same for $\mathbf{y}'$ and $\mathbf{z}'$

$$A_{\mathbf{y}'} = \begin{bmatrix} \mathbf{y}' \cdot \mathbf{x} \\ \mathbf{y}' \cdot \mathbf{y} \\ \mathbf{y}' \cdot \mathbf{z} \end{bmatrix} \quad A_{\mathbf{z}'} = \begin{bmatrix} \mathbf{z}' \cdot \mathbf{x} \\ \mathbf{z}' \cdot \mathbf{y} \\ \mathbf{z}' \cdot \mathbf{z} \end{bmatrix} .$$



Rotation Matrix

These three vectors are compactly represented as the three columns of a 3×3 matrix called the **rotation matrix**

$${}^A_B\mathbf{R} = \begin{bmatrix} \mathbf{x}' \cdot \mathbf{x} & \mathbf{y}' \cdot \mathbf{x} & \mathbf{z}' \cdot \mathbf{x} \\ \mathbf{x}' \cdot \mathbf{y} & \mathbf{y}' \cdot \mathbf{y} & \mathbf{z}' \cdot \mathbf{y} \\ \mathbf{x}' \cdot \mathbf{z} & \mathbf{y}' \cdot \mathbf{z} & \mathbf{z}' \cdot \mathbf{z} \end{bmatrix}$$

The subscript B and superscript A indicate that this matrix describes the rotation of **frame B with respect to frame A** . The order is important: ${}^A_B\mathbf{R} \neq {}^B_A\mathbf{R}$.



Representing a Frame with Respect to a Different Frame

In conclusion, a frame can be represented by:

- a vector to describe its origin
- a rotation matrix to describe its axes, i.e., its orientation.

This is the *pose* of the frame (position and orientation), or the pose of the rigid body (robot) attached to the frame.



Inverse Representation

What if we instead want ${}^B_A\mathbf{R}$?

Same reasoning, but we consider the projections of the axis $\mathbf{x}\mathbf{y}\mathbf{z}$ along the axis $\mathbf{x}'\mathbf{y}'\mathbf{z}'$

$${}^B_A\mathbf{R} = \begin{bmatrix} \mathbf{x} \cdot \mathbf{x}' & \mathbf{y} \cdot \mathbf{x}' & \mathbf{z} \cdot \mathbf{x}' \\ \mathbf{x} \cdot \mathbf{y}' & \mathbf{y} \cdot \mathbf{y}' & \mathbf{z} \cdot \mathbf{y}' \\ \mathbf{x} \cdot \mathbf{z}' & \mathbf{y} \cdot \mathbf{z}' & \mathbf{z} \cdot \mathbf{z}' \end{bmatrix}.$$

The dot product is commutative, and therefore ${}^B_A\mathbf{R} = {}^A_B\mathbf{R}^T$.



Change of Coordinates

Example: two robots seen in the beginning

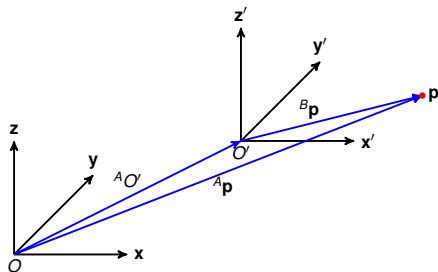
Problem statement: Given $A = O - \mathbf{x}\mathbf{y}\mathbf{z}$ and $B = O' - \mathbf{x}'\mathbf{y}'\mathbf{z}'$ and ${}^B\mathbf{p} = [p_x \ p_y \ p_z]^T$, determine ${}^A\mathbf{p}$.

We want to *change* the coordinates of $\mathbf{p}$ from frame B to frame A .



Change of Coordinates: case 1

Frames are parallel but do not have the same origin



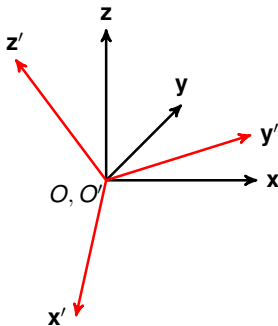
Let ${}^A O' = [\Delta x \ \Delta y \ \Delta z]^T$. Then coordinates of $\mathbf{p}$ in frame A are obtained by translation, i.e.,

$${}^A \mathbf{p} = {}^B \mathbf{p} + {}^A O' = \begin{bmatrix} p_x + \Delta x \\ p_y + \Delta y \\ p_z + \Delta z \end{bmatrix}$$



Change of Coordinates: case 2

Frames have the same origin but are not parallel



$${}^B\mathbf{p} = p_x\mathbf{x}' + p_y\mathbf{y}' + p_z\mathbf{z}'$$



Change of Coordinates: case 2

Recall:

$${}^A_B\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

Therefore

$${}^A\mathbf{x}' = r_{11}\mathbf{x} + r_{21}\mathbf{y} + r_{31}\mathbf{z}$$

$${}^A\mathbf{y}' = r_{12}\mathbf{x} + r_{22}\mathbf{y} + r_{32}\mathbf{z}$$

$${}^A\mathbf{z}' = r_{13}\mathbf{x} + r_{23}\mathbf{y} + r_{33}\mathbf{z}.$$



Change of Coordinates: case 2

Substitute into ${}^B\mathbf{p}$

$$\begin{aligned} {}^A\mathbf{p} &= p_x(r_{11}\mathbf{x} + r_{21}\mathbf{y} + r_{31}\mathbf{z}) + p_y(r_{12}\mathbf{x} + r_{22}\mathbf{y} + r_{32}\mathbf{z}) + p_z(r_{13}\mathbf{x} + r_{23}\mathbf{y} + r_{33}\mathbf{z}) \\ &= (p_x r_{11} + p_y r_{12} + p_z r_{13})\mathbf{x} + (p_x r_{21} + p_y r_{22} + p_z r_{23})\mathbf{y} + (p_x r_{31} + p_y r_{32} + p_z r_{33})\mathbf{z}. \end{aligned}$$

Recall definition of matrix-vector multiplication: ${}^A\mathbf{p} = {}^A_B\mathbf{R} {}^B\mathbf{p}$

To remember: superscripts and subscripts cancel out *diagonally*.



Example

Let ${}^B\mathbf{p} = [0 \ 2 \ 1]^T$ and

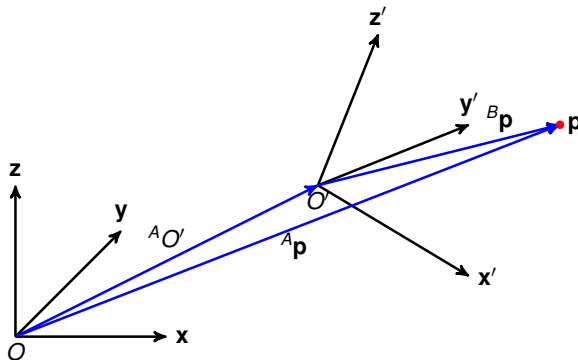
$${}^A_B\mathbf{R} = \begin{bmatrix} 0.70710678 & 0 & 0.70710678 \\ 0.61237244 & 0.5 & -0.61237244 \\ -0.35355339 & 0.8660254 & 0.35355339 \end{bmatrix}$$

To get coordinates of $\mathbf{p}$ with respect to A compute

$${}^A\mathbf{p} = {}^A_B\mathbf{R} {}^B\mathbf{p} = [0.70710678 \ 0.38762756 \ 2.0856042]^T.$$



Change of Coordinates: General Case



Must first translate (overlap origins) and then rotate (align axes)

$${}^A \mathbf{p} = {}^A_B \mathbf{R} {}^B \mathbf{p} + {}^A \mathbf{O}'$$



Example

Consider the previous figure. Assume ${}^B\mathbf{p} = [1 \ 4 \ 2]^T$. Let us furthermore assume that we have the description of frame B with respect to frame A , i.e., ${}^AO' = [7 \ -2 \ 1]^T$ and

$${}^A\mathbf{R} = \begin{bmatrix} 0.70710678 & 0 & 0.70710678 \\ 0.61237244 & 0.5 & -0.61237244 \\ -0.35355339 & 0.8660254 & 0.35355339 \end{bmatrix}$$

What is ${}^A\mathbf{p}$? Apply formula ${}^A\mathbf{p} = {}^A\mathbf{R}{}^B\mathbf{p} + {}^AO'$ obtaining ${}^A\mathbf{p} = [9.12132034 \ -0.61237244 \ 4.81765501]^T$.



Rotation Matrices

3×3 matrices subject to multiple constraints (*special orthogonal*)

- 1 each of its columns has length 1;
- 2 its columns are mutually orthogonal;
- 3 its determinant is 1.

Despite having 9 elements, they are defined by three parameters (more later)



Inverse of a Rotation Matrix

Theorem

The transpose of a rotation matrix is equal to its inverse, i.e., if $\mathbf{R}$ is a rotation matrix, then $\mathbf{R}^T = \mathbf{R}^{-1}$.

Proof: see whiteboard and lecture notes.

Combining theorem and previous result:

$${}^B_A\mathbf{R} = {}^A_B\mathbf{R}^T = [{}^A_B\mathbf{R}]^{-1}$$



Example

Rotation matrix expressing rotation of frame B with respect to A :

$${}^A_B\mathbf{R} = \begin{bmatrix} 0.9211 & -0.3894 & 0 \\ 0.3894 & 0.9211 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The matrix expressing the rotation of A with respect to B is its inverse, i.e., ${}^B_A\mathbf{R} = [{}^A_B\mathbf{R}]^{-1}$.
According to the theorem: it is ${}^A_B\mathbf{R}$

$${}^B_A\mathbf{R} = \begin{bmatrix} 0.9211 & 0.3894 & 0 \\ -0.3894 & 0.9211 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

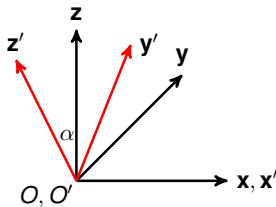


Elementary Rotation Matrices

General rotations can be obtained as composition of *simpler* rotations.

Assume two frames are coincident and then rotate one of the two about one common axis (move red frame $B = O' - \mathbf{x}'\mathbf{y}'\mathbf{z}'$ and keep black frame $A = O - \mathbf{xyz}$ fixed).

Start with rotation of angle α about $\mathbf{x}$



Elementary Rotation Matrices ($\mathbf{x}$ axis)

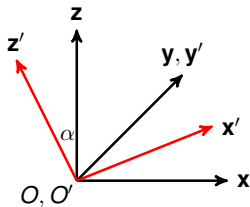
- $\mathbf{x}'$ did not move $\Rightarrow$ coordinates of the end point of axis $\mathbf{x}'$ in frame A are $[1 \ 0 \ 0]^T$
- coordinates of $\mathbf{y}'$ in frame A are $[0 \ \cos \alpha \ \sin \alpha]^T$.
- coordinates of $\mathbf{z}'$ in frame A are $[0 \ -\sin \alpha \ \cos \alpha]^T$.

Putting all together:

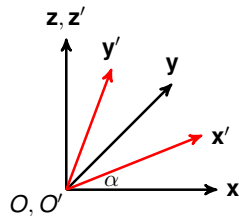
$$\mathbf{R}_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}$$



Elementary Rotation Matrices (**y** and **z** axes)



$$\mathbf{R}_y(\alpha) = \begin{bmatrix} \cos \alpha & 0 & \sin \alpha \\ 0 & 1 & 0 \\ -\sin \alpha & 0 & \cos \alpha \end{bmatrix}$$



$$\mathbf{R}_z(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

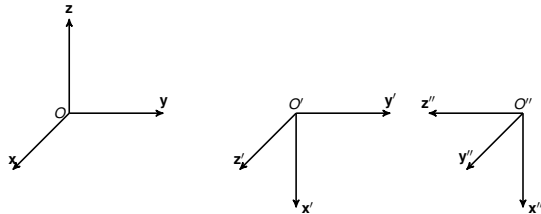
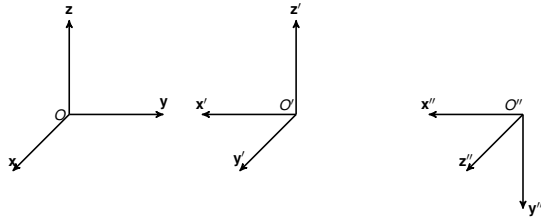


Composite Rotations

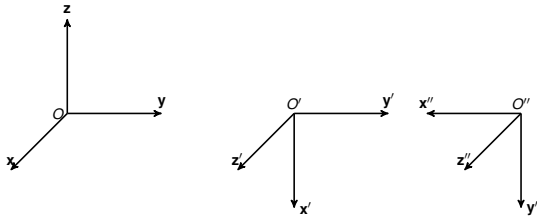
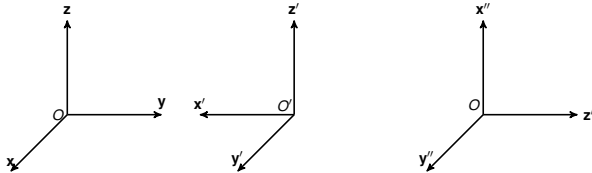
- rigid bodies can rotate about *arbitrary* axes
- generic rotations can be obtained composing elementary rotations
- important fact: **rotations do not commute**, i.e., the order matters
 - if a frame rotates first about $\mathbf{x}$ and then about $\mathbf{y}$, the result is different from rotating first about $\mathbf{y}$ and then about $\mathbf{x}$
- it is necessary to always specify about which axis rotations are performed, i.e., moving or fixed



Composite Rotations (about fixed axes)



Composite Rotations (about moving axes)



Rotation Matrices for Composite Rotations – Fixed Frame

- two initially aligned frames A and B ; A is fixed, B moves
- B rotates of angle α about $\mathbf{x}$ and then of an angle β about $\mathbf{z}$ where $\mathbf{x}$ and $\mathbf{z}$ are the axes of the fixed frame A .
- What is the expression for ${}^A_B\mathbf{R}$?

Solution:

- A and B are aligned; after first rotation we get an intermediate rotation matrix ${}^A_B\mathbf{R}' = \mathbf{R}_x(\alpha)$
- Rotating a matrix corresponds to rotate all its axes $\Rightarrow$ apply second rotation to axes of intermediate matrix

$${}^A_B\mathbf{R} = \mathbf{R}_z(\beta)\mathbf{R}_x(\alpha)$$



Rotation Matrices for Composite Rotations – Fixed Frame

General case:

- Let A and B be two initially coincident
- B undergoes a sequence of n rotations $\mathbf{R}_1, \mathbf{R}_2 \dots \mathbf{R}_n$, all expressed about the fixed frame A
- final expression:

$${}^A_B\mathbf{R} = \mathbf{R}_n \dots \mathbf{R}_2 \mathbf{R}_1$$

- **Important:** if rotations are about the fixed axes it is necessary to **premultiply**



Rotation Matrices for Composite Rotations – Moving Frame

General case:

- Let A and B be two initially coincident
- B undergoes a sequence of n rotations $\mathbf{R}_1, \mathbf{R}_2 \dots \mathbf{R}_n$, all expressed about the moving frame B
- final expression:

$${}^A_B\mathbf{R} = \mathbf{R}_1 \mathbf{R}_2 \dots \mathbf{R}_n.$$

- **Important:** if rotations are about the moving axes it is necessary to **postmultiply**



Example

Let A and B be two initially coincident frames. Frame B rotates first of α about the fixed axis $\mathbf{x}$, then β about the moving axis $\mathbf{z}$ and finally γ about the fixed axis $\mathbf{y}$.

Determine the final rotation matrix describing B in the frame A .

Answer (details on the whiteboard and on the lecture notes):

$${}^A_B\mathbf{B}''' = \mathbf{R}_y(\gamma){}_B^A\mathbf{B}'' = \mathbf{R}_y(\gamma)\mathbf{R}_x(\alpha)\mathbf{R}_z(\beta).$$



Composite Rotations

Theorem

Let ${}^A_B\mathbf{R}$ and ${}^B_C\mathbf{R}$ be two rotation matrices describing the orientation of frame B with respect to frame A and the rotation of frame C with respect to frame B, respectively. Then

$${}^A_C\mathbf{R} = {}^A_B\mathbf{R} {}^B_C\mathbf{R}$$

i.e., their product gives the orientation of frame C with respect to frame A.

Proof: See lecture notes (repeated change of coordinates on the columns of the second matrix).

Simple rule: cancellations occur diagonally

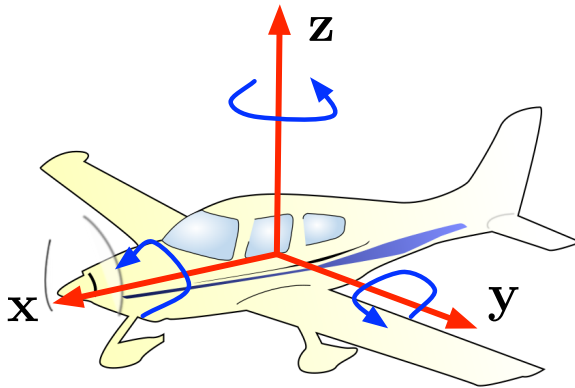


Rotation Parametrizations

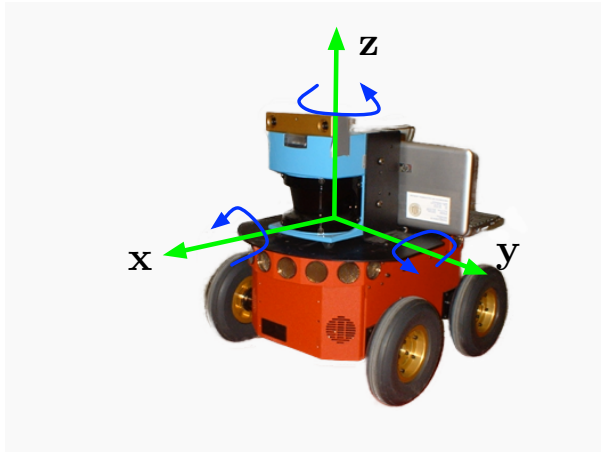
- Rotation matrices: 9 elements with many constraints $\Rightarrow$ fully determined by three parameters (i.e., three angles)
- many different choices, but only few used in practice. We will see just two:
 - Euler angles
 - Roll-Pitch-Yaw



Roll-Pitch-Yaw



Roll-Pitch-Yaw



Roll-Pitch-Yaw

- rotation α about the **x** axis (roll)
- followed by a rotation of β about the **y** axis (pitch)
- and then by a rotation of γ about the **z** axis (yaw)

Since all rotations are about the fixed frame,

$$\mathbf{R} = \mathbf{R}_z(\gamma)\mathbf{R}_y(\beta)\mathbf{R}_x(\alpha).$$



Euler Angles

- three successive rotations about the *moving axes*
- order: ZYZ
- α about Z, then β about Y, and γ about Z
- since all rotations are about the moving axes:

$$\mathbf{R} = \mathbf{R}_z(\alpha)\mathbf{R}_y(\beta)\mathbf{R}_z(\gamma)$$



Inverse problems

Given a desired rotation matrix $\mathbf{R}$ determine the three angles giving such rotation (either Euler or Roll-Pitch-Yaw)

Input:

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

Output: α, β, γ .



Inverse problems

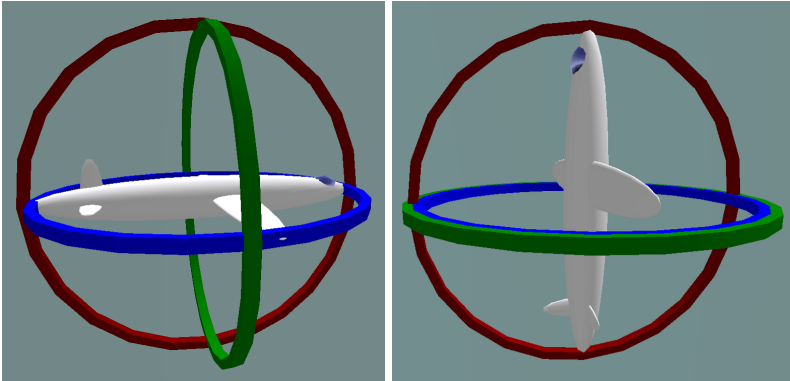
- Good news: there exist closed form solutions for either parametrization
- Multiple solutions (one to many situation). Why?
- Ex: for Euler one set of solutions is the following (see lecture notes for other set, and roll, pitch, yaw)

$$\alpha = \text{atan2}(r_{23}, r_{13}) \quad \beta = \text{atan2}\left(\sqrt{r_{13}^2 + r_{23}^2}, r_{33}\right) \quad \gamma = \text{atan2}(r_{32}, -r_{31})$$

- no need to recall them, but must know how to apply them
- in ROS there will be ready to use functions solving the problem for you



Gimbal Lock



Figures from https://en.wikipedia.org/wiki/Gimbal_lock



Gimbal Lock

Why does it happen? Let us explicitly write the rotation matrix for the ZYZ Euler angles

$$\mathbf{R} = \begin{bmatrix} \cos \alpha \cos \beta \cos \gamma - \sin \alpha \sin \gamma & -\cos \alpha \cos \beta \sin \gamma - \sin \alpha \cos \gamma & \cos \alpha \sin \beta \\ \sin \alpha \cos \beta \cos \gamma + \cos \alpha \sin \gamma & -\sin \alpha \cos \beta \sin \gamma + \cos \alpha \cos \gamma & \sin \alpha \sin \beta \\ -\sin \beta \cos \gamma & \sin \beta \sin \gamma & \cos \beta \end{bmatrix}$$

- If $\beta = k\pi$ then $\sin \beta = 0 \Rightarrow$ some terms are 0
- Inverse problem cannot be solved. Why? $\beta = k\pi$ implies first and third rotation axes are aligned, so final result is given by two rotations only.
- problem exists with every parametrization based on three angles
- problem also known as *singularity*

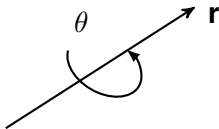


Quaternions

- alternative rotation parametrization that is not affected by singularities
- uses four parameters to represent one rotation
- complex algebraic structure pervasively used in robotics and computer graphics

Theorem (Euler's rotation theorem)

In three dimensions, every rotation is equivalent to a single rotation about an axis through the origin.



Quaternions

- similar to complex numbers, but with three *imaginary* components (i, j, k):
 $\mathbf{q} = a + bi + cj + dk$
- Properties of the imaginary components: $i^2 = j^2 = k^2 = ijk = -1$
- if $b = c = d = 0$ then we have a real number, and if $c = d = 0$ we have a complex number
- norm of a quaternion: $|\mathbf{q}| = \sqrt{a^2 + b^2 + c^2 + d^2}$
- *unit quaternion*: a quaternion of length 1

Why do we care about quaternions? ROS represents rotations using quaternions (not rotation matrices)



From quaternion to matrices

Let $\mathbf{q} = a + bi + cj + dk$ be a unit quaternion. Its associated rotation matrix is (some times written $\mathbf{R}(\mathbf{q})$)

$$\mathbf{R} = \begin{bmatrix} 2(a^2 + b^2) - 1 & 2(bc - ad) & 2(bd + ac) \\ 2(bc + ad) & 2(a^2 + c^2) - 1 & 2(cd - ab) \\ 2(bd - ac) & 2(cd + ab) & 2(a^2 + d^2) - 1 \end{bmatrix}$$

Exercise: verify that it is indeed a rotation matrix.



From quaternions to axis angle

Given a unit quaternion $\mathbf{q} = a + bi + cj + dk$ it is interesting to determine its parameters according to Euler's theorem.

- angle: $\theta = 2 \arccos a$.
- components of the rotation axis $\mathbf{r} = [r_1 \ r_2 \ r_3]^T$

$$r_1 = \frac{b}{\sin \frac{\theta}{2}} \quad r_2 = \frac{c}{\sin \frac{\theta}{2}} \quad r_3 = \frac{d}{\sin \frac{\theta}{2}}$$

Not defined when $\sin \frac{\theta}{2} = 0$, i.e., when $\theta = 0$ or $\theta = 2\pi \Rightarrow$
makes sense because for these two values there is no rotation at all.



From axis-angle to quaternions

Given θ and a unitary vector $\mathbf{r} = [r_1 \ r_2 \ r_3]^T$ the associated quaternion is

$$\mathbf{q} = \cos \frac{\theta}{2} + i \sin \frac{\theta}{2} r_1 + j \sin \frac{\theta}{2} r_2 + k \sin \frac{\theta}{2} r_3.$$



Non-uniqueness

- Mapping between quaternions and rotations is not one-to-one
- $\mathbf{q}$ is a unit quaternion $\Rightarrow -\mathbf{q}$ is a unit quaternion
- $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation matrix $\mathbf{R}$ (prove it!)
- as per Euler's theorem, a rotation of θ about $\mathbf{r}$ is equivalent to a rotation of $2\pi - \theta$ about $-\mathbf{r}$



From rotation matrices to quaternions

Given a rotation matrix $\mathbf{R}$, determine the equivalent unit quaternion $\mathbf{q} = a + bi + cj + dk$

$$a = \frac{\sqrt{r_{11} + r_{22} + r_{33} + 1}}{2} \quad b = \frac{r_{32} - r_{23}}{4a} \quad c = \frac{r_{13} - r_{31}}{4a} \quad d = \frac{r_{21} - r_{12}}{4a}.$$



Quaternion algebra and rotating points with quaternions

- Given two quaternions $\mathbf{q}_1$ and $\mathbf{q}_2$ it is immediate to define the sum and product operation treating them as polynomials
- given $\mathbf{q} = a + bi + cj + dk$, its *conjugate* is $\mathbf{q}^* = a - bi - cj - dk$
- let $\mathbf{p} = [p_x \ p_y \ p_z]^T$ be a point $\Rightarrow \mathbf{p}' = 0 + p_x i + p_y j + p_z k$
- let $\mathbf{q}$ be a unit quaternion
- $\mathbf{p}'' = \mathbf{q}\mathbf{p}'\mathbf{q}^* = p'' + p''_x i + p''_y j + p''_z k$
- the vector $[p''_x \ p''_y \ p''_z]$ is the vector obtained by rotating $\mathbf{p}$ by the rotation associated with $\mathbf{q}$
- important: interpolation between orientations



Homogeneous Coordinates

- points are identified by vectors in $\mathbb{R}^3$
- directions can also be identified by a (unary) vector in $\mathbb{R}^3$
- **homogenous coordinates** offer a unified way to represent and distinguish both points and directions
 - will lead to convenient **transformation matrices**
- idea: represent both points and directions using a *four dimensional* vector



Homogeneous Coordinates

- a point $\mathbf{p} = [x \ y \ z]^T$ is represented by the four dimensional vector
 $\mathbf{p} = [x \ y \ z \ 1]^T$
 - last coordinate is **always** 1
 - use same symbol for both Cartesian and homogeneous coordinates
- a direction $\mathbf{d} = [x \ y \ z]^T$ is represented by the four dimensional vector
 $\mathbf{d} = [x \ y \ z \ 0]^T$
 - last coordinate is **always** 0
- so last element allows to distinguish points from directions



Example

Orientation of frame $B - \mathbf{x}'\mathbf{y}'\mathbf{z}'$ with respect to frame $A - \mathbf{xyz}$ is given by the rotation matrix:

$${}^A_B\mathbf{R} = \begin{bmatrix} 0.5721 & 0.0064 & 0.8202 \\ 0.5721 & 0.7135 & -0.4046 \\ -0.5878 & 0.7006 & 0.4045 \end{bmatrix}$$

- Each column represents the direction of one axis of B in frame A
- direction $\mathbf{x}'$ in homogeneous coordinates is $\mathbf{x}' = [0.5721 \ 0.5721 \ -0.5878 \ 0]^T$



Transformation Matrices

Definition

Let $A - \mathbf{xyz}$ and $B - \mathbf{x'y'z'}$ be two frames. Let ${}^A\mathbf{p} = [p_x \ p_y \ p_z]^T$ be the coordinates of the origin of $B - \mathbf{x'y'z'}$ expressed in frame A , and let ${}^A_B\mathbf{R}$ be the rotation matrix expressing the orientation of frame B with respect to A . Then the *transformation matrix* expressing frame $B - \mathbf{x'y'z'}$ with respect to frame $A - \mathbf{xyz}$ is the 4×4 matrix defined as follows:

$${}^A_B\mathbf{T} = \begin{bmatrix} & {}^A_B\mathbf{R} & & p_x \\ & & & p_y \\ & & & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

- last row is **always** $[0 \ 0 \ 0 \ 1]$



Transformation Matrices

- many interpretations
- first three columns: directions of $\mathbf{x}'\mathbf{y}'\mathbf{z}'$ in homogeneous coordinates
- last column: coordinates of $\mathbf{p}$ in homogeneous coordinates
- **Interpretation #1**: single object combining all elements necessary to represent a frame (rotation and origin)



Transformation Matrices

- **Interpretation #2:** operator to transform points and directions expressed in homogeneous coordinates

Let $\mathbf{p}$ be a point

$$\mathbf{T}\mathbf{p} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11}p_x + r_{12}p_y + r_{13}p_z + t_x \\ r_{21}p_x + r_{22}p_y + r_{23}p_z + t_y \\ r_{31}p_x + r_{32}p_y + r_{33}p_z + t_z \\ 1 \end{bmatrix}$$

- $\mathbf{p}$ expressed in homogeneous coordinates and result expressed in homogeneous coordinates
- rotate $\mathbf{p}$ by $\mathbf{R}$ and then translate by $[t_x \ t_y \ t_z]^T$
- can express rotation, translation, and rototranslation



Transformation Matrices

Let $\mathbf{d} = [d_x \ d_y \ d_z \ 0]^T$ is a direction expressed in homogeneous coordinates.

$$\mathbf{T}\mathbf{d} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} d_x \\ d_y \\ d_z \\ 0 \end{bmatrix} = \begin{bmatrix} r_{11}d_x + r_{12}d_y + r_{13}d_z \\ r_{21}d_x + r_{22}d_y + r_{23}d_z \\ r_{31}d_x + r_{32}d_y + r_{33}d_z \\ 0 \end{bmatrix}$$

- result is rotated direction in homogeneous coordinates
- no translation applied (*directions cannot be translated*)



Transformation Matrices

Special transformation matrices (as operators)

$$\mathbf{T}(\Delta_x, \Delta_y, \Delta_z) = \begin{bmatrix} 1 & 0 & 0 & \Delta_x \\ 0 & 1 & 0 & \Delta_y \\ 0 & 0 & 1 & \Delta_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{T}(\mathbf{z}, \theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{T}(\mathbf{x}, \theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{T}(\mathbf{y}, \theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Transformation Matrices

- **Interpretation #3**: operator to change coordinates
- let ${}^A_B\mathbf{T}$ is the transformation matrix representing frame B with respect to frame A
- let ${}^B\mathbf{p}$ is the vector expressing the coordinates of $\mathbf{p}$ in frame B

$${}^A\mathbf{p} = {}^A_B\mathbf{T} {}^B\mathbf{p}.$$

- result follows from previous two interpretations



Transformation Matrices

- Interpretation #4: operator to transform frames
- consider ${}^A\mathbf{T}_B$ and ${}^B\mathbf{T}_C$
- can multiply them together (dimensions agree)

$$\begin{aligned} {}^A\mathbf{T}_C &= {}^A\mathbf{T}_B {}^B\mathbf{T}_C = \begin{bmatrix} {}^A\mathbf{R} & \begin{matrix} {}^A O'_x \\ {}^A O'_y \\ {}^A O'_z \end{matrix} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} {}^B\mathbf{R} & \begin{matrix} {}^B O''_x \\ {}^B O''_y \\ {}^B O''_z \end{matrix} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \\ &= \begin{bmatrix} {}^A\mathbf{R} {}^B\mathbf{R} & {}^A\mathbf{R} {}^B O'' + {}^A O' \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} {}^A\mathbf{R} & {}^A O'' \\ 0 & 0 & 0 & 1 \end{bmatrix} = {}^A\mathbf{T}_C \end{aligned}$$

Rotate directions and rototranslates origin



Transformation Matrices: Chaining and Inverse

- Previous relation can be applied multiple times in sequence

$${}^0_n\mathbf{T} = {}^0_1\mathbf{T} {}^1_2\mathbf{T} {}^2_3\mathbf{T} \dots {}^{n-2}_{n-1}\mathbf{T} {}^{n-1}_n\mathbf{T}.$$

- Inverse problem: given ${}^A_B\mathbf{T}$ we are interested in ${}^B_A\mathbf{T}$
 - usual requirement: ${}^A_B\mathbf{T} {}^B_A\mathbf{T} = {}^B_A\mathbf{T} {}^A_B\mathbf{T} = \mathbf{I}$
- solution via simple linear algebra

$${}^A_B\mathbf{T} = \begin{bmatrix} {}^A_B\mathbf{R} & {}^A O' \\ 0 & 0 & 0 & 1 \end{bmatrix} \Rightarrow {}^B_A\mathbf{T} = \begin{bmatrix} {}^B_A\mathbf{R} & -{}^B_A\mathbf{R} {}^A O' \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Example: Change of Coordinates

Consider case seen in slide 5 (two robots observing the same object).

- A world frame, B green frame, C red frame
- ${}^A_B\mathbf{T}$ and ${}^A_C\mathbf{T}$ are known
- left ${}^B\mathbf{p}$ pose of object as observed by left robot. What is the pose of the object with respect to the right robot ${}^C\mathbf{p}$?

1 First Solution

- 1 determine ${}^A\mathbf{p} = {}^A_B\mathbf{T}{}^B\mathbf{p}$
- 2 then determine ${}^C_A\mathbf{T}$ inverting ${}^A_C\mathbf{T}$
- 3 overall solution: ${}^C\mathbf{p} = {}^C_A\mathbf{T}{}^A_B\mathbf{T}{}^B\mathbf{p}$

2 Second Solution

- 1 compute ${}^C_B\mathbf{T}$ (how?)
- 2 overall solution: ${}^C\mathbf{p} = {}^C_B\mathbf{T}{}^B\mathbf{p}$



Transformation Trees

- useful data structure to quickly perform arbitrary changes of coordinates
- supported by ROS via the `tf/tf2` packages

Definition

Let $\mathcal{A} = \{A_1, A_2, \dots, A_n\}$ be a set of n frames and let $\mathcal{T} = \{ {}^{A_{i(1)}}_{A_{j(1)}}\mathbf{T} \dots {}^{A_{i(k)}}_{A_{j(k)}}\mathbf{T} \}$ be a set of k transformation matrices between the given frames. The *transformation graph* defined by these frames and transformation matrices is a graph $G = (V, E)$ with n vertices and k edges. Each vertex is labeled with one of the frames, i.e., v_1 is labeled A_1 , v_2 is labeled A_2 , and so on. Edge $(v_l, v_m) \in E$ if and only if ${}^{A_l}_{A_m}\mathbf{T} \in \mathcal{T}$.

Definition (Transformation tree)

A *transformation tree* is a transformation graph with no cycles.

Transformation Trees

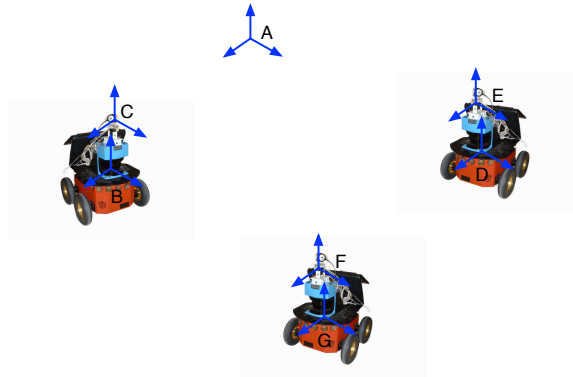
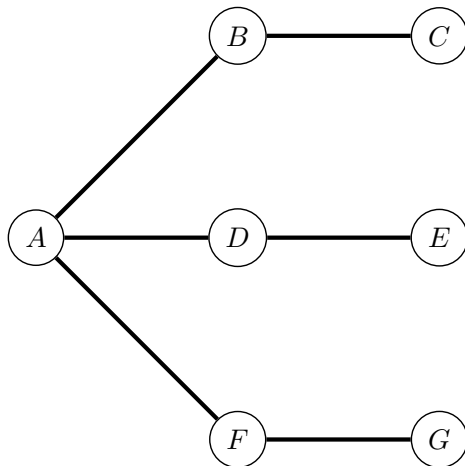


Figure: Three robots with 7 frames.



Transformation Trees



Transformation tree based on $\mathcal{T} = \{ {}^A_B\mathbf{T}, {}^B_C\mathbf{T}, {}^A_D\mathbf{T}, {}^D_E\mathbf{T}, {}^A_F\mathbf{T}, {}^F_G\mathbf{T} \}$



Transformation Trees

Using transformation trees:

- given ${}^A\mathbf{p}$ can I determine ${}^B\mathbf{p}$?
- if ${}^A_B\mathbf{T}$ is in the tree, the answer is obviously *yes*
- if there is a path between A and B in the tree, then ${}^A_B\mathbf{T}$ can be computed and the answer is *yes*
- if there is no path between A and B in the tree, the answer is *no*

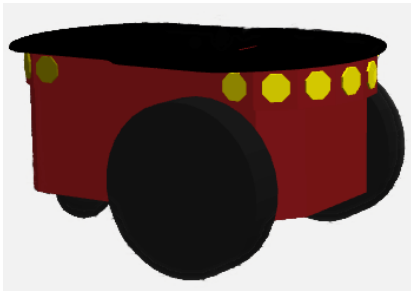


Kinematic Motion Models

- focus on wheeled vehicles
- useful to anticipate the effect of actions (we will need this in planning and navigation)
- related to exported API in many ROS nodes
- Question 1: if I give a certain input, what is the output?
- Question 2: if I desire a certain output, which input should I give?



Differential Drive

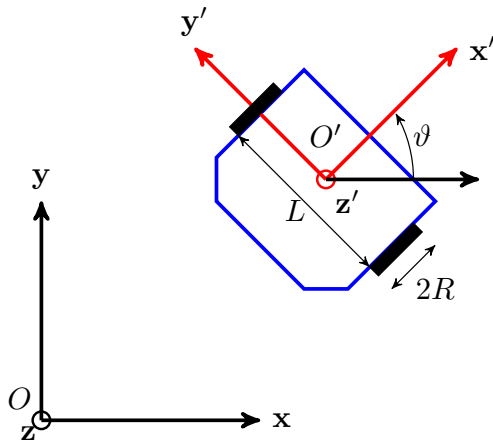


Differential Drive

- two or four actuated wheels
- wheels on the same side are *coupled*, i.e., they receive the same control
- relevant parameters and inputs:
 - R : radius of the wheels
 - L : separation between the wheels
 - ω_R, ω_L : angular velocities for the right and left wheel
- What type of motions can it perform?
 - 1 forward/backward straight
 - 2 turn in place
 - 3 motion along an arc



Differential Drive



Frame $B = O' - \mathbf{x}'\mathbf{y}'\mathbf{z}'$ is attached to the robot. Pose is x, y, ϑ .

Differential Drive Transformation Matrix

$${}^A_B\mathbf{T} = \begin{bmatrix} \cos \vartheta & -\sin \vartheta & 0 & x \\ \sin \vartheta & \cos \vartheta & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Fully specified by the pose x, y, ϑ .

In some instances we write just x, y, ϑ instead of ${}^A_B\mathbf{T}$ (one-to-one correspondence).



Differential Drive Transition Equations

$$\begin{aligned}\dot{x} &= \frac{R}{2}(\omega_R + \omega_L) \cos \vartheta \\ \dot{y} &= \frac{R}{2}(\omega_R + \omega_L) \sin \vartheta \\ \dot{\vartheta} &= \frac{R}{L}(\omega_R - \omega_L)\end{aligned}$$

Note: two inputs (ω_R, ω_L) , but three state variables (x, y, ϑ)

Often more convenient thinking in terms of translational and rotational velocities

That is what you normally do in ROS



Example

Let $R = 0.05m$ and $L = 0.15m$. Determine ω_L and ω_R so that the translational speed is $0.1m/s$ and the rotational speed is $0.05rad/s$.

From the above relationships the (positive) translational speed can be written as

$$\begin{aligned}\sqrt{\dot{x}^2 + \dot{y}^2} &= \sqrt{\left(\frac{R}{2}(\omega_R + \omega_L) \cos \vartheta\right)^2 + \left(\frac{R}{2}(\omega_R + \omega_L) \sin \vartheta\right)^2} \\ &= \sqrt{\frac{R^2}{4}(\omega_R + \omega_L)^2} = \frac{R}{2}(\omega_R + \omega_L).\end{aligned}$$

We therefore obtain a linear system with two equations in two unknowns:

$$\frac{R}{2}(\omega_R + \omega_L) = 0.1 \quad \frac{R}{L}(\omega_R - \omega_L) = 0.05$$

The system solves to $\omega_R = 2.075rad/s$ and $\omega_L = 1.925rad/s$.



Approximate Equations – Discrete Time Models

When ω_R, ω_L are given:

$$x(t + \Delta t) \approx x(t) + \dot{x}\Delta t = x(t) + \frac{R}{2}(\omega_R + \omega_L) \cos \vartheta(t) \Delta t$$

$$y(t + \Delta t) \approx y(t) + \dot{y}\Delta t = y(t) + \frac{R}{2}(\omega_R + \omega_L) \sin \vartheta(t) \Delta t$$

$$\vartheta(t + \Delta t) \approx \vartheta(t) + \dot{\vartheta}\Delta t = \vartheta(t) + \frac{R}{L}(\omega_R - \omega_L) \Delta t$$



Approximate Equations – Discrete Time Models

When $v_t, v_r \neq 0$ are given:

$$\begin{aligned}x(t + \Delta t) &\approx x(t) + \left[-\frac{v_t(t)}{v_r(t)} \sin \vartheta(t) + \frac{v_t(t)}{v_r(t)} \sin(\vartheta(t) + v_r(t)\Delta t) \right] \\y(t + \Delta t) &\approx y(t) + \left[\frac{v_t(t)}{v_r(t)} \cos \vartheta(t) - \frac{v_t(t)}{v_r(t)} \cos(\vartheta(t) + v_r(t)\Delta t) \right] \\\vartheta(t + \Delta t) &\approx \vartheta(t) + v_r(t)\Delta t\end{aligned}$$

If instead $v_r = 0$, the above relationships simplify leading to a straight motion (note that these relationships are not approximated):

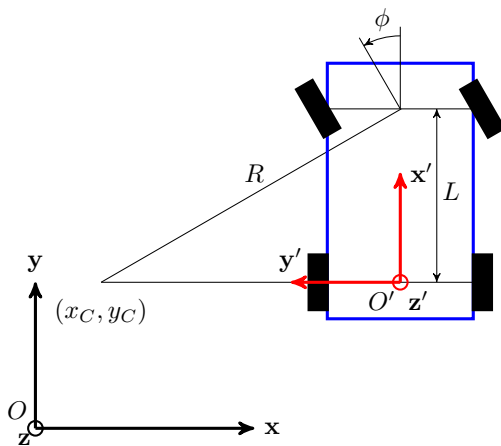
$$\begin{aligned}x(t + \Delta t) &= x(t) + v_t \sin \vartheta(t) \\y(t + \Delta t) &= y(t) + v_t \cos \vartheta(t) \\\theta(t + \Delta t) &= \theta(t).\end{aligned}$$



Skid steer drive



Ackerman Steer



Inputs: v_t and ϕ . Radius of the center of rotation: $R = \frac{L}{\tan \phi}$



Ackerman Steer Kinematic Model

Continuous time model:

$$\dot{x} = v_t \cos \vartheta$$

$$\dot{y} = v_t \sin \vartheta$$

$$\dot{\vartheta} = \frac{v_t}{L} \tan \phi$$

Discrete time model

$$x(t + \Delta t) \approx x(t) + v_t \Delta t \cos \vartheta$$

$$y(t + \Delta t) \approx y(t) + v_t \Delta t \sin \vartheta$$

$$\vartheta(t + \Delta t) \approx \vartheta(t) + \frac{v_t \Delta t}{L} \tan \phi$$



Velocity

- so far we have mostly talked about static poses
- for time varying poses, the derivative with respect to time leads to velocities
- position component: simple
- orientation: slightly more complicated – we will consider just special cases (see next)



Kinematics in ROS

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced

<https://sites.ucmerced.edu/scarpin>

<https://robotics.ucmerced.edu>



Kinematics in ROS

- various packages devoted to kinematics
- `geometry_msgs` offers messages to represent and process quantities related to kinematics
- `tf2` implements transformation trees (and more)



ROS messages for Kinematics

Message Type	Quantity
<code>geometry_msgs::msg::Point</code>	Point in $\mathbb{R}^3$
<code>geometry_msgs::msg::PointStamped</code>	Point with time stamp
<code>geometry_msgs::msg::Quaternion</code>	Orientation
<code>geometry_msgs::msg::QuaternionStamped</code>	Orientation with time stamp
<code>geometry_msgs::msg::Pose</code>	Position and orientation (as quat.)
<code>geometry_msgs::msg::PoseStamped</code>	Pose with time stamp
<code>geometry_msgs::msg::Pose2D</code>	Pose in the plane, i.e., x, y, θ
<code>geometry_msgs::msg::Transform</code>	Transformation matrix
<code>geometry_msgs::msg::TransformStamped</code>	Transformation matrix with t.s.
<code>geometry_msgs::msg::Vector3</code>	Direction in space
<code>geometry_msgs::msg::Vector3Stamped</code>	Direction in space with time stamp
<code>geometry_msgs::msg::Twist</code>	Velocity (linear and angular)
<code>geometry_msgs::msg::TwistStamped</code>	Velocity with time stamp

Table: ROS messages to represent geometric data.



Time Stamps

- many messages have *stamped* version (e.g., `Vector3` and `Vector3Stamped`)
- useful for quantities changing over time
- stamped version includes a `std_msgs/Header` message in the beginning, e.g., `Vector3Stamped` is

```
std_msgs/Header header  
Vector3 vector
```



Headers

- stamp: temporal time stamp
- frame_id: name of the frame with respect to which this quantity is referred to

```
# Two-integer timestamp that is expressed as seconds and nanoseconds  
builtin_interfaces/Time stamp
```

```
# Transform frame with which this data is associated.  
string frame_id
```



geometry_msgs::msg::Pose2D

- convenience message type introduced specifically for robots moving on a plane (e.g., differential drive or skid-steer.)

```
float64 x  
float64 y  
float64 theta
```

- however, some mobile robots broadcast their pose as `geometry_msgs::msg::Pose`.
- **use** `ros2 topic list` to determine the format of the pose



Controlling a Differential Drive in ROS

- messages of type `geometry_msgs::msg::Twist` are used to specify velocities (e.g., to `cmd_vel` topics)

```
#This expresses velocity in free space broken into  
# its linear and angular parts.
```

```
Vector3  linear  
Vector3  angular
```

Recall how frames are attached to differential drives: `linear.x` is the translational speed, `angular.z` is the rotational speed

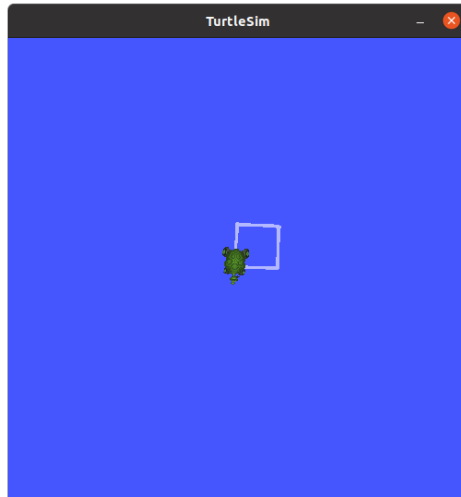


Controlling a Differential Drive

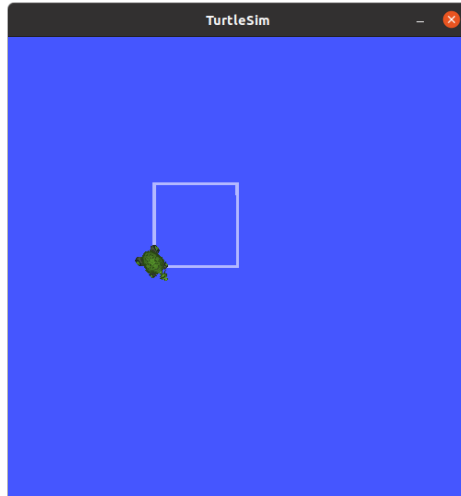
- Drawing a square in open loop: `drawsquare.cpp` ☹
- Drawing a square in closed loop: `drawsquarefb.cpp` ☺



The problem with open loop control



(Simple) closed loop control



A first glimpse into topic remapping

- `turtlesim` is controlled as a differential drive. Can I use the same code to control a differential drive robot?
- Yes, and it is not even necessary to recompile the code, thanks to *remapping*

```
ros2 run examples drawsquare --ros-args --remap  
  turtle1/cmd_vel:=p2dx/cmd_vel
```

or

```
ros2 run examples drawsquare --ros-args --remap  
  turtle1/cmd_vel:=p3at/cmd_vel
```



The transform library (package)

- two versions used, `tf` and `tf2` (the latter phasing out the former)
- Provides:
 - classes supporting geometric/kinematic concepts
 - static functions to perform geometric/kinematic operations
 - tools to track multiple coordinate frames changing over time (transform trees)
 - names standardization for common frames
- package `tf2_msgs` provides two messages: `TF2Error` and `TFMessage`



tf2 classes

Class	Description
<code>tf2::Matrix3x3</code>	Rotation Matrix
<code>tf2::Quaternion</code>	Quaternion
<code>tf3::Vector3</code>	Point or vector
<code>tf2::Transform</code>	Rigid transformation, i.e., rotation and translation

Table: `tf2` classes to represent geometric data.



tf2::msg::TFMessage

- Structure of tf2::msg::TFMessage

```
geometry_msgs/TransformStamped[] transforms
```

- Structure of geometry_msgs::msg::TransformStamped

```
#The frame id in the header is used as the reference frame  
# of this transform.
```

```
std_msgs/Header header
```

```
#The frame id of the child frame to which  
# this transform points.
```

```
string child_frame_id
```

```
#Translation and rotation in 3-dimensions of child_frame_id  
# from header.frame_id
```

```
Transform transform
```



tf2::msg::TFMessage (cont'd)

- Structure of `geometry_msgs::msg::Transform`

This represents the transform between two coordinate frames in

Vector3 translation

Quaternion rotation

- ${}^B_A\mathbf{T}$ will be represented by a message `TransformStamped` with `child_frame_id` set to *A* and `frame_id` set to *B*.



tf2::msg::TF2Error

```
uint8 NO_ERROR = 0
uint8 LOOKUP_ERROR = 1
uint8 CONNECTIVITY_ERROR = 2
uint8 EXTRAPOLATION_ERROR = 3
uint8 INVALID_ARGUMENT_ERROR = 4
uint8 TIMEOUT_ERROR = 5
uint8 TRANSFORM_ERROR = 6

uint8 error
string error_string
```



Quaternions and Rotations in ROS

- `geometry_msgs` **does not** provide a message for rotation matrices
 - rotations are always sent as quaternions
- `tf2::Quaternion` represents a quaternion
 - not a duplicate: the message has just data, the class includes useful methods, too
 - `getAngle` returns the angle associated with the quaternion
 - `getAxis` returns the axis associated with the quaternion
 - `setEulerZYZ` sets the quaternion to the rotation associated with a given triplet of Euler angles
 - `setRPY` sets the quaternion to the rotation associated with a given triplet of roll-pitch-yaw angles



Quaternions and Rotations in ROS

Useful static functions

- `tf2::getYaw`: accepts as parameter an instance of `tf2::Quaternion` and returns the associated yaw angle. This is extremely useful for mobile robots moving in the plane.
- `tf2::toMsg`: converts an instance of `tf2::Quaternion` into a message of type `geometry_msgs::Quaternion` so that it can be published (see also listing ?? for more details.)
- `tf2::fromMsg`: converts a message of type `geometry_msgs::Quaternion` into the equivalent class of the `tf2` package.

Example: `republishpose.cpp`



Quaternions and Rotations in ROS

- `tf2::Matrix3x3` represents a rotation matrix
- `setRotation` converts a quaternion into a rotation matrix
- `getRotation` returns the quaternion equivalent to a rotation matrix
- various other methods to compute inverse, transpose, etc, (see documentation)
- Examples: `geom.cpp` and `republishpose.cpp`



Frames in ROS

- exchanged as messages of type `geometry_msgs/TransformStamped`
- stamped because they can change over time
- two strings: `frame_id` in the header and `child_frame_id` in the message
 - ${}^B_A\mathbf{T}$ will be represented by a message `TransformStamped` with `child_frame_id` set to *A* and `frame_id` set to *B*.



Transformation Trees in ROS

- handled by `tf2`
- allows to look at transformation *buffered in time*, i.e., at present time or in the past (up to 10 secs; configurable).
- to take advantage of the infrastructure, nodes must listen to or *broadcast* transformations
- often you can take advantage by just listening

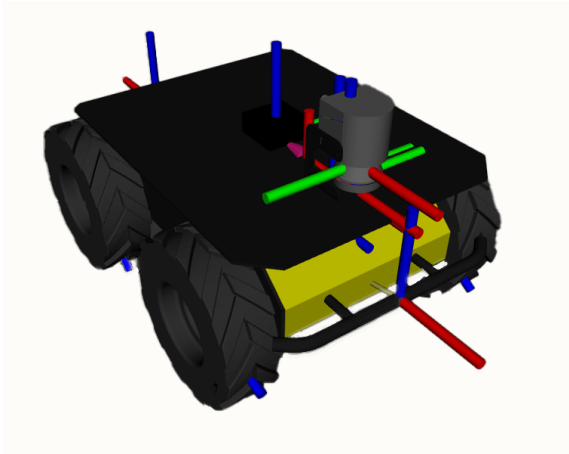


Transformation Trees in ROS

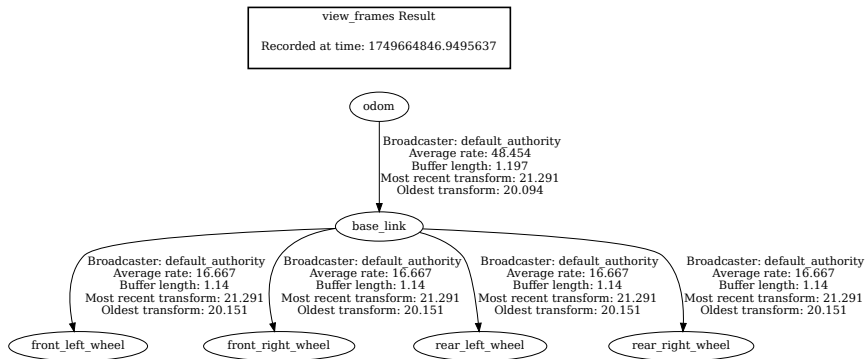
- Frames are exchanged through two topics: `/tf` and `/tf_static`
- as the name suggests, `/tf_static` is used for transformations that do not change over time
- Example: placement of frames on a robot



Transformation Trees in ROS



Transformation Trees in ROS



Interfacing with tf2

- use object of type `TransformListener` and `TransformBroadcaster`
- can also subscribe directly to topics, but more complex

- Example: `tf_listener.cpp`

- To lookup in the past:

```
rclcpp::Time  
pastlookup=nodeh->get_clock()->now()-tf2::durationFromSec(3);
```

- can also lookup transformation between arbitrary frames, provided the tree is connected

- Example: `tf_broadcaster.cpp`

- Visualizing the transformation tree:

```
ros2 run tf2_tools view_frame.py
```



Broadcasting frames

transforms:

- header:

stamp:

sec: 1673894532

nanosec: 133698111

frame_id: base_link

child_frame_id: myframe

transform:

translation:

x: 4.0

y: 0.0

z: 2.0

rotation:

x: 0.0

y: 0.0

z: 0.0

w: 1.0



Standard Frames in ROS

- `base_link` : rigidly attached to the robot (typically center of mass); x points forward, z points up;
 - `odom` : world fixed frame; pose expressed in this frame is continuous and can drift over time;
 - `map` : world fixed frame; pose expressed in this frame can be discontinuous and should not drift over time;
 - `earth` : world frame with origin at the center of the earth; used only in special cases
- *world frames* do not change over time



ROS Programming – Advanced Topics

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced

<https://sites.ucmerced.edu/scarpin>

<https://robotics.ucmerced.edu>



Remapping: changing node names

- process to change some standard arguments in ROS applications
- example: node name. Why would you change it?
 - having multiple instances of the same node with the same name is allowed, but will most likely create problems
- `ros2 run talklisten talker --ros-args --remap __name:=newtalker`
- the output will be

```
[INFO] [1673935144.419936102] [newtalker]: Sending message #1
[INFO] [1673935145.409056027] [newtalker]: Sending message #2
[INFO] [1673935146.409073411] [newtalker]: Sending message #3
[INFO] [1673935147.409061516] [newtalker]: Sending message #4
[INFO] [1673935148.408931236] [newtalker]: Sending message #5
```



Remapping: changing topic names

- often times the functionality of a node is the same, but just the topic name changes

```
ros2 run packagename executablename --ros-args --remap  
originaltopicname:=newtopicname
```

- we already saw this in chapter 4



Namespaces

- introduced to avoid name clashes (like in C++ and other languages)
- example: a robot with two cameras runs two instances of the same node to process the two separate input streams. How can we separate the output streams, too?
- **namespaces** allow to solve this problem without changing the source
- namespaces allow to change the names of resources (nodes names, topics, etc) by prepending a unique suffix that can be specified when the executable is started

```
ros2 run talklisten talker --ros-args --remap __ns:=/t1
```

- now `talker` publishes to a topic called `/t1/message` (even though in the source we call it `message`)



Namespaces (cont'd)

- if we now run the listener, it will not print anything b/c it subscribes to `message`
- this can be fixed by remapping the listener, too

```
ros2 run talklisten listener --ros-args --remap  
message:=/t1/message
```

- remappings can be specified in launch files, too (example: `t1remap.launch.xml`)



ROS names

- having two resources with exactly the same name is legit, but will likely create problems
- Example: we can run two instance of `talker` from the `demo_nodes_cpp` package
- `ros2 node list` tells us this is a bad idea

WARNING: Be aware that are nodes in the graph that share an exact name, this can have unintended side effects.

```
/talker
```

```
/talker
```

- How can we fix it?
- Remapping the name with `-remap __name` is a partial fix because they continue to publish to the same topic (may or may not be what we want)



ROS names

- the solution is to remap the namespaces for one of the nodes
- `ros2 run demo_nodes_cpp talker -ros-args -remap __ns:=/t2`
- `ros2 node list` gives
 - `/t2/talker`
 - `/talker`
- `ros2 node list` give
 - `/chatter`
 - `/t2/chatter`
- Success! Now they are fully separated



ROS fully qualified names

- ROS resources (topics, service, etc.) have *relative* and *fully qualified names*
- a **fully qualified name** starts with / and identifies the namespace within which the resource exist
- relative names are referred to the namespace of the nodes that create them.
- relative names *do not* start with /
- if a resource is create without setting a namespace, its fully qualified name is the name of the resource preceded by the / character
- if a resource is created within a namespace, its fully qualified name is the name of the resource preceded by the name of the namespace
- Exercise: go back and revisit the listings in chapter 3



Parameters

- names and namespaces are associated with all nodes
- ROS parameters allow the user to define new application-specific parameters
- Ex: on which port is a sensor connected? With which frequency should a sensor be queried?
- We may wish to set them when a node starts, or even change them during execution
- `rclcpp` provides methods to set, retrieve, and change parameters from within an executable
- parameters can also be modified from the command line
- `ros2 param list` shows all parameters associated with running nodes



Parameters

- start both the `talker` and `listener` nodes from the `demo_nodes_cpp` and run `ros2 param list`

- output

```
/listener:
qos_overrides./parameter_events.publisher.depth
qos_overrides./parameter_events.publisher.durability
qos_overrides./parameter_events.publisher.history
qos_overrides./parameter_events.publisher.reliability
start_type_description_service
use_sim_time
/talker:
qos_overrides./parameter_events.publisher.depth
qos_overrides./parameter_events.publisher.durability
qos_overrides./parameter_events.publisher.history
qos_overrides./parameter_events.publisher.reliability
start_type_description_service
use_sim_time
```

- all nodes have a parameter called `use_sim_time` (more later)



Interacting with parameters from the command line

- **use** `ros2 param describe` to get information about an existing parameter
- `ros2 param describe talker use_sim_time`
- **output:**

Parameter name: `use_sim_time`

Type: `boolean`

Constraints:

- **use** `ros2 param get` to retrieve the value of an existing parameter
 - `ros2 param get talker use_sim_time`
 - **output:**
- Boolean value is: `False`
- parameters are local and not global (each node has its own copy of `use_sim_time`)

Interacting with parameters from the command line

- the value of a parameter can also be set from the command line
- use `ros2 param set` to retrieve the value of an existing parameter
- `ros2 param set talker use_sim_time True`
- verify that the parameter has changed!
- general syntax

```
ros2 param set node_name parameter_name value
```



Interacting with parameters via `rclcpp`

- In your code, you first declare a parameter and then you can retrieve its value
- parameters have a name (string) and a type that must be specified when they are declared; default values **must** be given
- Code example: `paramclient.cpp`
- parameters can be changed from the outside in various ways, e.g. when an executable starts
- `ros2 run examples paramclient -ros-args -p "sensorport" := "/dev/USB0"`
- parameters can also be changed from the command line
- `ros2 param set paramclient sensorport /dev/USB1`
- if you use `ros2 param set`, parameters must be set **after** the node is running
- try it and you'll see the value changes



More about parameters

- Parameters can also be specified in launch files
- Example: `param.launch.xml`
- nodes can also change parameters using `set_parameter_value` (mirrors `get_parameter_value`)



Loading and saving parameters

- All parameters can be saved to a YAML file that can then be edited, if needed
`ros2 param dump <nodename>`
- Parameters can also be loaded at run time after a node starts
`ros2 param load <nodename> <parameterfile>`
- we can load the parameters from a YAML file when the node starts, e.g.,
`ros2 run talklisten listener -ros-args -params-file
parameters.yaml`
- the same can be specified in the launch file, see e.g., `paramfile.launch.xml`



Intercepting parameters changes on-the-fly

- First example performs busy waiting to get the parameter – not practical
- to be notified when a parameter changes, subscribe to the topic
`parameter_events` of type `rcl_interfaces/msg/ParameterEvent`



rcl_interfaces/msg/ParameterEvent

The time stamp when this parameter event occurred.

builtin_interfaces/Time stamp

Fully qualified ROS path to node.

string node

New parameters that have been set for this node.

Parameter[] new_parameters

Parameters that have been changed during this event.

Parameter[] changed_parameters

Parameters that have been deleted during this event.

Parameter[] deleted_parameters

Intercepting parameters changes on-the-fly

- Example: `paramevent.cpp`



The parameter `use_sim_time`

- **all nodes** have a parameter called `use_sim_time`
- useful when the node interacts with a simulator (e.g., Gazebo)
- if `use_sim_time` is `True`, the node retrieves time from the simulator and not from the system clock
- useful when a node must reason about time (e.g., when using `tf2`)



Calling Services

- we already saw the commands `ros2 service list` and how `ros2 interface` can be used to learn about a service
- For example `ros2 interface show sensor_msgs/srv/SetCameraInfo` shows

```
sensor_msgs/CameraInfo camera_info # The camera_info to store
---
bool success                        # True if the call succeeded
string status_message              # Used to give details about success
```

- example: `servicecall.cpp`



OOP in ROS

- all code written so far used instances of `rclcpp::Node`
- legit but does not scale to larger applications
- **problem**: logic governing nodes is not associated with the node itself (we had to use global variables)
- solution: inherit from the class `rclcpp::Node` and use encapsulation
- Example: `talker_oop.cpp`
- later on we'll see how to trigger producers from timers
- Example: `listener_oop.cpp`



Nav2

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced
<https://sites.ucmerced.edu/scarpin>
<https://robotics.ucmerced.edu>



ROS Actions

- non-blocking, preemptable function calls
 - **non-blocking**: control returns to the caller
 - **pre-emptable**: execution can be interrupted before it terminates
- contrast with *services*
- extensively used in the ROS 2 navigation stack (aka Nav2)
- completing some operations (e.g., navigating to a point) may take a long time



Key Concepts

Action Server : the node offering and implementing an action.

Action Client : the node sending a request to an action server.

Goal : the message sent by the client to the server when an action is initiated

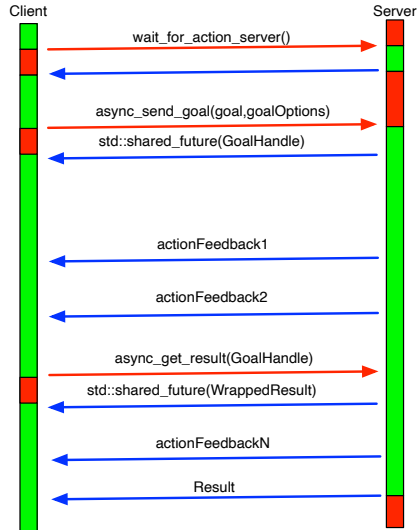
Feedback : a message periodically sent back by the server to the client

Result : a unique message sent by the server back to the client once the action is completed (e.g, success or failure.)

- busy waiting is also possible (but discouraged)
- support for actions is provided by the package `rclcpp_action`



Logic Flow



Relevant Functions

`wait_for_action_server` : waits until the action server is ready to receive a goal.
Can accept a timeout parameter to limit the wait time.

`action_server_is_ready` : checks if the action server is ready to accept a goal request.

`async_send_goal` : sends a goal to the action server and registers callback functions

`async_get_result` : gets the result for an active goal specified through a goal handle returned by `async_send_goal`. The function returns a shared future to a `WrappedResult` holding the result.

`async_cancel_goal` : pre-empts the action by canceling the current goal. The function returns a future that is set when the cancel request is honored.



More about actions

- must provide three callback functions (one for the goal, one for the feedback, one for the result)
- `spin` functions must be called to trigger the callback functions
- actions proceed asynchronously as a separate thread and are built on top of the C++ concurrency support library
- many functions related to actions return a C++ *future*



Futures

- action clients and servers run concurrently
- implemented on top of C++ concurrency library
- results often returned as instances of `std::shared_future`.
- through a future, a client can verify if the server has completed its computation and provided a result
- call `spin_until_future_complete` to spin until the result expected to be passed back through a future is ready (**blocking**)



Goals, Goal Options, Goal Handles and Wrapped Results

- `async_send_goal` requires a `Goal` and `SendGoalOptions`
- Goal information can be obtained with `ros2 interface show`
- `SendGoalOptions` features three pointers to the callback functions
- `async_send_goal` returns a future to `GoalHandle`
- the future to `GoalHandle` can be passed to `async_get_result` to get a future to `WrappedResult`
- when `WrappedResult` is ready, one can extract the result
- or one can skip this altogether and just do everything via the callback functions



Example: spinning the Turtlebot in place

- action of type `nav2msgs::action::Spin`; inspect its structure with `ros2 interface show`
- provided by the navigation stack, rotates the robot in place (has nothing to do with the `spin` function)
- goal consists of `target_yaw` and `time_allowance` (an instance of `Duration`)
- see `actioncall.cpp` for full example

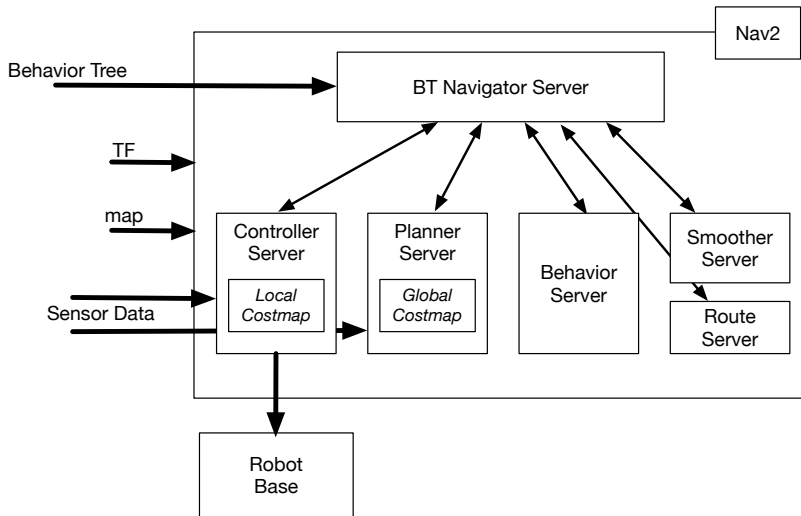


Planning in ROS

- *Nav2* provides both planning and navigation
- integrates perception and replanning; implements many of the concepts seen so far
- necessitates of various other nodes in place (e.g., to localize the robot, get information about surroundings, etc.)
- does not implement open loop planning, but rather integrates sensors and feedback



Overall Nav2 architecture



Nav2 inputs and outputs

Inputs

- a behavior tree specifying how to orchestrate the interactions among submodules;
- transformations from $\mathbb{t} \mathbb{f} 2$, which describe the kinematic arrangement of objects in the environment (including the current pose of the robot), as well as the configuration of sensors mounted on the robot;
- a map of the environment in which the robot is operating;
- sensor data from the robot's onboard systems, particularly sensors capable of measuring distances to obstacles.

Outputs: a stream of commands sent to the robot's actuators



Nav2 Servers

Planner: the planner module is in charge of computing a **global plan** between assigned start and goal poses. The computed plan is informed by the available map of the environment, and by the kinematic model and shape of the robot. The output of the planner is an *open loop* reference path. This module is implemented by the planner server.

Controller: the controller module follows a path computed by the planner (aka **local plan**). It does so by issuing velocity commands to the underlying platform. In doing so the controller implements a *closed loop* strategy, i.e., it continuously monitors the onboard sensors to ensure the robot follows the assigned path. This module is implemented by the controller server.



Nav2 Servers

Smoother: the smoother modifies a path produced by the planner to account for additional criteria that the planner may have not accounted for. This module is implemented by the smoother server.

Behavior Server: implements maneuvers to handle failures or unforeseen events (e.g., when the robot gets stuck and cannot make progress). The behaviors are triggered by the BT Navigation Server when the robot fails to make progress toward its assigned goal. Currently, implements three maneuvers: *spin*, *wait*, and *back up*.

Route Server: high-level planner that computes paths on a navigation graph. This differs from the Planner Server, which computes paths based on the environment map, although the navigation graph may have been extracted from the environment map.

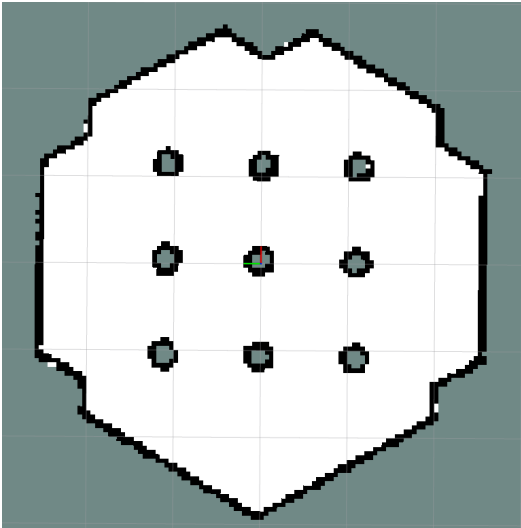


Localization, Maps, Cost Maps

- Nav2 **must** know the current pose of the robot within its environment
- localization is provided by a localization algorithm matching sensor readings to an a priori map of the environment
- ... therefore a map of the environment is needed, too.
- **global cost map**: built on top of the map; used by the planner server for global planning
- **local cost map**: dynamically built based on sensor inputs; used by the controller server for local planning
- the global is provided through the `/map` topic, and is an instance of `nav_msgs::msg::OccupancyGrid`



Map



Layers

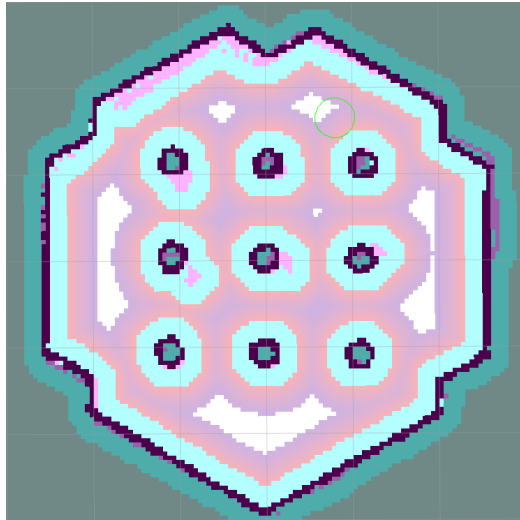
Static Layer: Represents the static map of the environment retrieved from `/map` at startup.

Obstacle Layer: Incorporates information about static and dynamic obstacles detected by the onboard sensors. This layer handles dynamic components that were not modeled in the static layer.

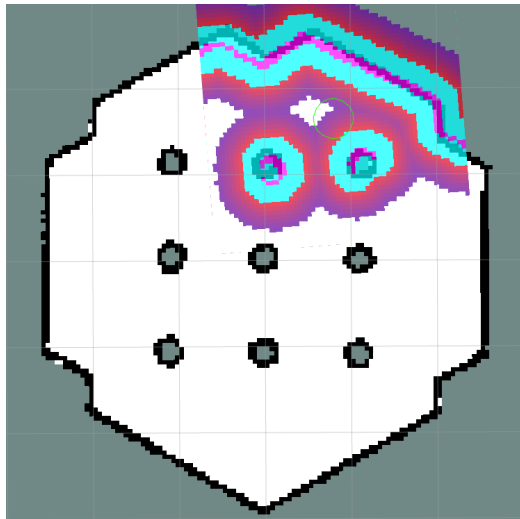
Voxel Layer: Similar to the obstacle layer, but relies on sensors providing three-dimensional data, such as depth cameras or three-dimensional range finders.

Inflation Layer: The inflation layer adds a safety margin by increasing the costs of cells surrounding the obstacles (thus inflating them). Encourages the planner to stay clear of obstacles when possible. The inflation layer retrieves its data from the other layers. The width of the inflation can be configured using a parameter called `inflation_radius`, which should be chosen based on the size of the robot.

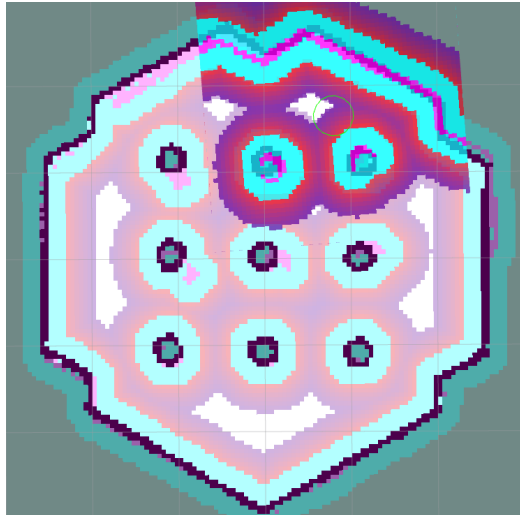
Global cost map



Local cost map



Global and local cost map



Maps and costmaps

- handled by the `nav2_costmap_2d` package, and they are all instances of `nav_msgs::msg::OccupancyGrid`.
- bidimensional regular grids, where each cell contains a cost value between 0 and 255.
 - 0: free space
 - 253: obstacles
 - 254: *lethal obstacles* (aka forbidden region)
 - 255: unknown
 - Intermediate values are also used, with higher values guiding the planner to prefer paths that maintain a safe clearance from obstacles

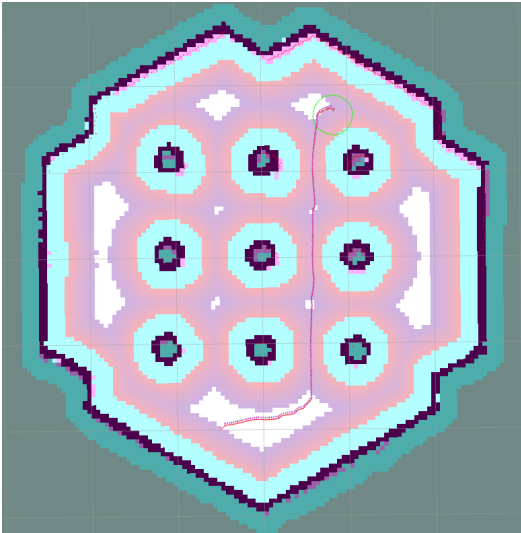


Planner Server

- can be linked to many plugins
- we will use the NavfnPlanner plugin
- compute a path from the current cell to a goal cell
- how? build a navigation function assigning costs to cells using the costs in the global cost map
- grid cells (states) are explored using either Dijkstra or A*
- produces a path, but **does not** send velocity commands to the robot



Global cost map

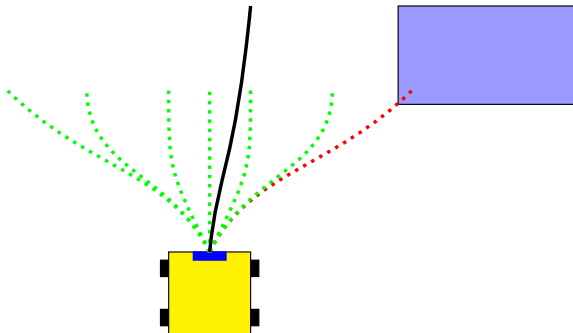


Controller Server

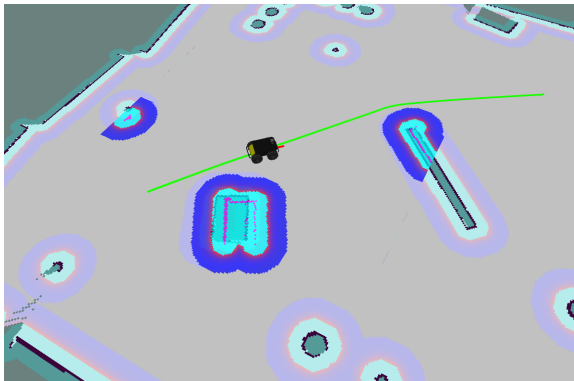
- implemented by the node `controller_server` through the action `follow_path`.
- sends velocity commands to the robot trying to match the global path produced by the planner server (an instance of `nav::msg::Path`)
- in charge of reacting to unforeseen changes and errors (using the local cost map)
- can be linked to many plugins
- we will use the DWB plugin
 - generates set of candidate trajectories (uses forward kinematic models equations)
 - scores each trajectory and pick the best one



DWB Planner



Global and Local Plans



BT Navigation Server – Default Behavior

- alternates between *navigation* and *recovery*
- **navigation**
 - compute global path and updated it (replan)
 - pass global path to the controller server that tracks one part of it
 - if either server fails, switch to *contextual recovery* (check if goal changed and clear maps). If this step fails, switch to recovery.
- **recovery**: cycle through; when one succeeds go back to navigation
 - clear costmaps
 - spin
 - wait
 - backup



Planning

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced

<https://sites.ucmerced.edu/scarpin>

<https://robotics.ucmerced.edu>

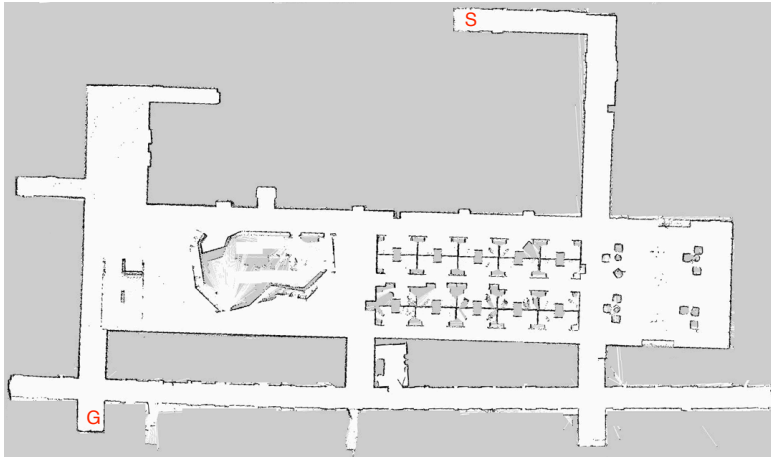


Planning definition

- **Planning**: the problem of selecting a sequence of actions to achieve an assigned goal
 - not a unique definition accepted by everyone
- often formulated in terms of state spaces
- goal must be specified, but starting location may or may not be assigned
- **Plan**: a sequence of actions or a function determining which action to take (e.g., based on current state, current time, uncertainty, etc...)
- so *planning* is the process, and *plan* is the result
- planning is where AI meets robotics...



Prototypical Planning Problem



Go from *S* to *G*

Three Different Planning Problems: Deterministic Planning

- **Deterministic planning**: given a start state $\mathbf{x}_s$, a goal state $\mathbf{x}_g$ (or a set of goal states), and a *state transition equation* $f(\mathbf{x}, \mathbf{u})$, determine a sequence of inputs $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_n$ to transform the start state in the goal state.

$$\mathbf{x}_s = \mathbf{x}_0 \xrightarrow{\mathbf{u}_1} \mathbf{x}_1 = f(\mathbf{x}_0, \mathbf{u}_1) \xrightarrow{\mathbf{u}_2} \mathbf{x}_2 = f(\mathbf{x}_1, \mathbf{u}_2) \xrightarrow{\mathbf{u}_3} \dots \quad \mathbf{x}_n = f(\mathbf{x}_{n-1}, \mathbf{u}_n) = \mathbf{x}_g$$

- the sequence $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_n$ is the plan
- *deterministic* means that the outcome of each action is fully predictable
- problem reduces to *graph search*



Three Different Planning Problems: Feedback Planning

- **Feedback Planning**: given a start state x_s and a goal state x_g and the objective is to determine a feedback function (also called *policy*) $\pi : X \rightarrow U$ defining for each state the action to take.
- implicitly assumes state is known (see how π is defined)
- considers the case where the outcome of actions is unpredictable
 - must know which action to execute from *any* possible state
- usually solved using a different formalism, i.e., Markov Decision Processes



Three Different Planning Problems: Planning in Belief Spaces

- **Planning in belief spaces**: given x_s and x_g the hypothesis of state observability no longer holds (current state cannot be determined with certainty)
- most generic case
- outcome of actions is not predictable and state is not precisely known
- must formulate plans based on uncertain information (belief)
- **significantly harder** than the previous two cases (we will not cover it).



Discrete Models

- state space set is discrete, finite, and known a priori: $S = \{x_1, x_2, \dots, x_n\}$
- For each state $x_i \in S$, a finite set of actions (or inputs) $U(x)$ is defined
 - $U(x)$ is the set of actions that can be executed in state x
 - different states may have different set of actions
 - set of all actions: $U = \cup_{x \in S} U(x)$
 - with a slight abuse of notation, often write U instead of $U(x)$
 - U is also finite



Features of Planning Algorithms

Completeness : a planning algorithm is *complete* if it will always find a solution, if one exists

Uninformed : a planner that has only access to the problem structure (states, input, start/goal)

Informed : a planner that uses additional information to expedite the planning process (e.g., domain specific heuristics about how *good* a state is)



Open Loop Planning

Builds upon discrete model, adding a time invariant state transition equation

$$x_t = f(x_{t-1}, u_t) \quad x_i \in S, u_t \in U(x_{t-1}).$$

Alternative formulation:

$$K = \{(x, u) \in S \times U \mid x \in S \wedge u \in U(x)\}$$

Then $f : K \rightarrow S$

Open loop planning must be used with caution...



The Planning Graph

For the above problem definition, a *planning graph* can be defined. It is a **directed** graph $G = (V, E)$ defined as follows:

- 1 Vertices set V : one vertex per state
 - 2 Edges set E : if there exists $u \in U(x_i)$ such that $x_j = f(x_i, u)$, then we add a directed edge from x_i to x_j
- edges set is one-to-one to with K

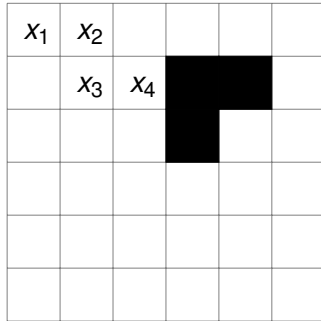
Definition

Let $G = (V, E)$ be a graph, and $x_1 \in V$, $x_n \in V$ be two vertices. A **path** p between x_1 and x_n is a sequence of vertices $x_1, x_2, \dots, x_n$ such that $(x_i, x_{i+1}) \in E$ for $1 \leq i \leq n - 1$.

One-to-one mapping between paths and plans.



Example: grid world



Unweighted Graph Search/Planning Problems

Directed Graph Search Problem: *Given a directed graph $G = (V, E)$, and vertices $x_s \in V$, $x_g \in V$, determine a path from x_s to x_g or return failure if no path can be found.*

Not a YES/NO problem (if a path exists, it must be returned)



Weighted Graph Search/Planning Problems

Definition

Let $G = (V, E)$ be a weighted graph, and let $c : E \rightarrow \mathbb{R}$ be its cost function. Let $p = x_1, x_2, \dots, x_n$ be a path in G . The **cost of path** p is the sum of the costs of its edges, i.e.,

$$c(p) = \sum_{i=1}^{n-1} c(x_i, x_{i+1}).$$

Weighted Directed Graph Search Problem: *Given a directed weighted graph $G = (V, E)$ with non-negative cost function $c : E \rightarrow \mathbb{R}_{\geq 0}$, and vertices $x_s \in V$, $x_g \in V$, determine a path of minimum cost from x_s to x_g or return failure if no path can be found.*

Minimum cost is unique, but path of minimum cost is not necessarily unique.



Common Traits in Discrete Planning Algorithms

We will see four different planning problems. They all share the following:

- all algorithms start from x_s
- vertices to be processed are stored in a data structure that is often referred to as the *OPEN* list or queue
 - may or may not be prioritized
- after a node is removed from the *OPEN* data structure (**expanded**), it is moved into a container data structure called *CLOSED*
 - *CLOSED* can at times be removed, but it is useful to reason about the correctness
- *parent* of a vertex: backpointer to the vertex from which it was discovered; *null* if it has not been discovered yet;
- *visited*: binary flag to mark vertices that have not yet been discovered; useful to avoid re-visiting vertices (could also use *parent* for the same purpose)
- *spanning tree*: tree rooted at x_s showing how the graph has been visited



Breadth First Search – BFS

```
Data:  $G = (V, E)$ ,  $x_s \in V$ ,  $x_g \in V$   
Result: Path from  $x_s$  to  $x_g$  if it exists, or FAILURE  
1 foreach  $x \in V$  do  
2      $x.parent \leftarrow \text{null};$   
3      $x.visited \leftarrow \text{false};$   
4  $OPEN.initializeEmpty();$   
5  $CLOSED.initializeEmpty();$   
6  $OPEN.insert(x_s);$   
7  $x_s.visited \leftarrow \text{true};$   
8 while not  $OPEN.empty()$  do  
9      $x \leftarrow OPEN.remove();$   
10     $CLOSED.insert(x);$   
11    foreach  $x' \in V$  such that  $(x, x') \in E$  do  
12        if  $x'.visited = \text{false}$  then  
13             $x'.visited \leftarrow \text{true};$   
14             $x'.parent \leftarrow x;$   
15            if  $x' = x_g$  then  
16                return  $\text{ExtractPath}(x_s, x_g);$   
17            else  
18                 $OPEN.insert(x');$   
19 return FAILURE;
```

Algorithm 1: BFS/DFS algorithm

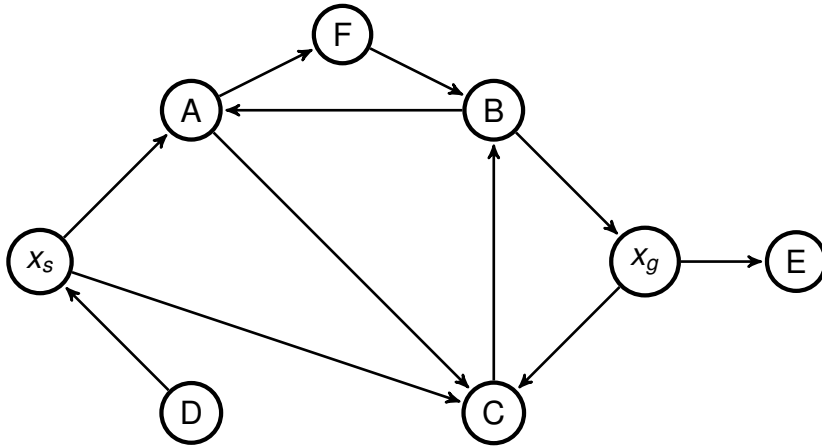


BFS Features

- *OPEN* is a FIFO container (queue)
- complexity: $\mathcal{O}(|V| + |E|)$
- finds a shortest plan in terms of actions (every action/edge costs the same)
- ExtractPath walks its way backwards from x_g to x_s via the backpointers
- BFS is complete



Example

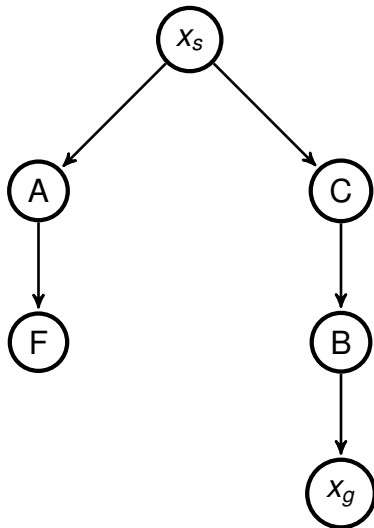


Example

Step	$OPEN$	x_s	A	B	C	D	E	F	x_g
0	x_s	N/T	N/F	N/F	N/F	N/F	N/F	N/F	N/F
1	A,C	N/T	x_s/T	N/F	x_s/T	N/F	N/F	N/F	N/F
2	C,F	N/T	x_s/T	N/F	x_s/T	N/F	N/F	A/T	N/F
3	F,B	N/T	x_s/T	C/T	x_s/T	N/F	N/F	A/T	N/F
4	B	N/T	x_s/T	C/T	x_s/T	N/F	N/F	A/T	N/F
5	$\emptyset$	N/T	x_s/T	C/T	x_s/T	N/F	N/F	A/T	B/T



Rooted Tree (BFS)



Depth First Search (DFS)

- identical pseudocode, but *OPEN* is now a LIFO (stack)
- DFS is complete when graph is finite
- complexity: $\mathcal{O}(|V| + |E|)$
- cannot say anything about the length of the computed plan

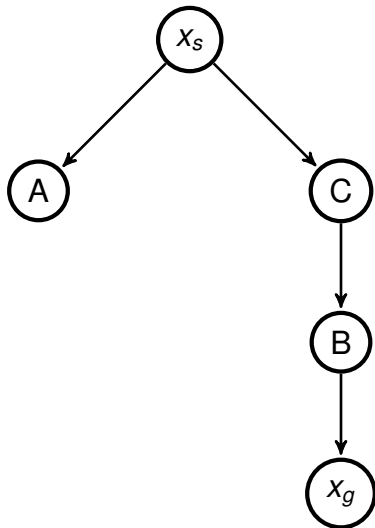


Example

Step	<i>OPEN</i>	x_s	A	B	C	D	E	F	x_g
0	x_s	N/T	N/F	N/F	N/F	N/F	N/F	N/F	N/F
1	A,C	N/T	x_s /T	N/F	x_s /T	N/F	N/F	N/F	N/F
2	B,A	N/T	x_s /T	C/T	x_s /T	N/F	N/F	N/F	N/F
3	A	N/T	x_s /T	C/T	x_s /T	N/F	N/F	N/F	B/T



Rooted Tree(DFS)



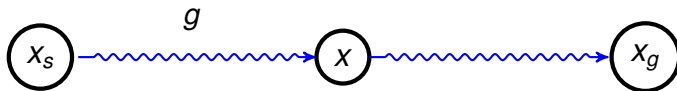
Dijkstra's Algorithm

- solves more realistic case of weighted graphs
 - returns a path of **minimal cost**
- implemented inside some of ROS planners
- developed around ideas widely used in many other planning algorithms
- still an *uninformed* algorithm



Key Ideas

- for each vertex, a **cost to come** is determined (indicated as g)
 - it is the cost to come from the source vertex x_s
- costs are iteratively refined (lowered)
 - *label correcting algorithm*
- $g(x)$ is the lowest cost to come from x_s to x discovered *so far*
- shortest/optimal path *must* be composed by optimal/shortest subpaths



Key Ideas and Algorithmic Details

- start exploring from x_s
- OPEN is a priority queue prioritized by the cost to come g
- as the graph is explored, look for *shortcuts*, i.e, ways to decrease g
- once the optimal cost of a vertex is determined, freeze it and do not consider the vertex anymore



Dijkstra's Algorithm

Data: $G = (V, E)$, $x_s \in V$, $x_g \in V$, $c : E \rightarrow \mathbb{R}_{\geq 0}$

Result: Shortest path from x_s to x_g if it exists, or FAILURE

```
1 foreach  $x \in V$  do
2    $x.g \leftarrow \infty$ ;  $x.parent \leftarrow \text{null}$ ;
3  $x_s.g \leftarrow 0$ ;
4  $OPEN.initializeEmpty()$ ;
5  $OPEN.insert(x_s)$ ;
6  $CLOSED.initializeEmpty()$ ;
7 while not  $OPEN.empty()$  do
8    $x \leftarrow OPEN.remove()$ ;
9    $CLOSED.insert(x)$ ;
10  if  $x = x_g$  then
11    return  $\text{ExtractPath}(x_s, x_g)$ ;
12  foreach  $x' \in V$  such that  $(x, x') \in E$  do
13    if  $x'.g = \infty$  then
14       $x'.parent \leftarrow x$ ;
15       $x'.g \leftarrow x.g + c(x, x')$ ;
16       $OPEN.insert(x')$ ;
17    else if  $x.g + c(x, x') \leq x'.g$  then
18       $x'.g \leftarrow x.g + c(x, x')$ ;
19       $x'.parent \leftarrow x$ ;
20       $OPEN.rebalance(x')$ ;
21 return FAILURE;
```



Properties

- when x_g is removed from OPEN, an optimal path has been found. Why?
- every time a node is rediscovered, an attempt is made to lower its costs
- must rebalance OPEN because cost to go of nodes changes
- **Key difference** with BFS/DFS: Dijkstra's algorithm does not terminate when x_g is discovered for the first time, but when x_g is removed from OPEN (compare pseudocode)



Correctness and Complexity

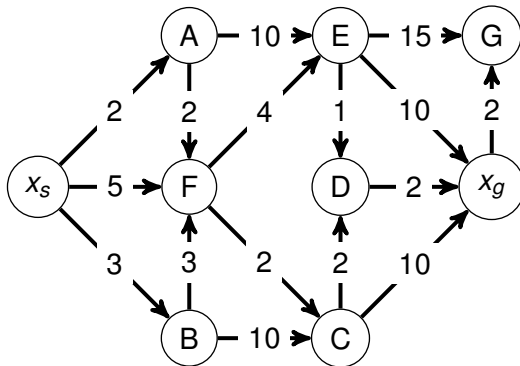
- computational complexity depends on the data structure used to implement OPEN
 - e.g., $\mathcal{O}((V + E) \log V)$ if using a binary heap or $\mathcal{O}(V \log V + E)$ if using a Fibonacci heap

Theorem

If a path between x_s and x_g does not exist, Dijkstra's algorithm returns FAILURE. If a path between x_s and x_g exists, Dijkstra returns a path of minimum cost, as per the path cost formerly defined.



Example



Example

Step	OPEN	x_s	A	B	C	D	E	F	G	x_g
0	$x_s/0$	$N/0$	N/∞	N/∞	N/∞	N/∞	N/∞	N/∞	N/∞	N/∞
1	$A/2, B/3, F/5$	$N/0$	$x_s/2$	$x_s/3$	N/∞	N/∞	N/∞	$x_s/5$	N/∞	N/∞
2	$B/3, F/4, E/12$	$N/0$	$x_s/2$	$x_s/3$	N/∞	N/∞	$A/12$	$A/4$	N/∞	N/∞
3	$F/4, E/12, C/13$	$N/0$	$x_s/2$	$x_s/3$	$B/13$	N/∞	$A/12$	$A/4$	N/∞	N/∞
4	$C/6, E/8$	$N/0$	$x_s/2$	$x_s/3$	$F/6$	N/∞	$F/8$	$A/4$	N/∞	N/∞
5	$D/8, E/8, x_g/16$	$N/0$	$x_s/2$	$x_s/3$	$F/6$	$C/8$	$F/8$	$A/4$	N/∞	$C/16$
6	$E/8, x_g/10$	$N/0$	$x_s/2$	$x_s/3$	$F/6$	$C/8$	$F/8$	$A/4$	N/∞	$D/10$
7	$x_g/10, G/23$	$N/0$	$x_s/2$	$x_s/3$	$F/6$	$C/8$	$F/8$	$A/4$	$E/23$	$D/10$
8	$G/23$	$N/0$	$x_s/2$	$x_s/3$	$F/6$	$C/8$	$F/8$	$A/4$	$E/23$	$D/10$



Exploration Tree

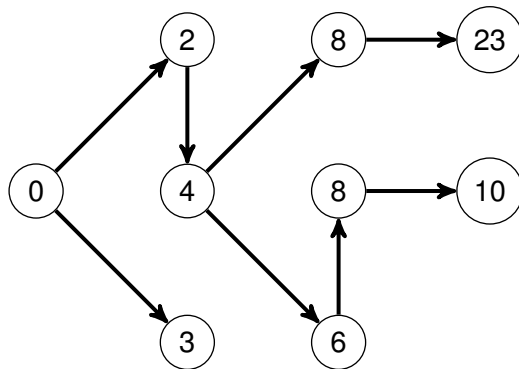


Figure: Tree produced by Dijkstra's algorithm.



Single Source Shortest Path

```
Data:  $G = (V, E)$ ,  $x_s \in V$ ,  $c : E \rightarrow \mathbb{R}_{\geq 0}$   
Result: Shortest from  $x_s$  to each vertex in  $V$  reachable from  $x_s$   
1 foreach  $x \in V$  do  
2    $x.parent \leftarrow \text{null}$ ;  
3    $x.g \leftarrow \infty$ ;  
4  $x_s.g \leftarrow 0$ ;  
5  $OPEN.initializeEmpty()$ ;  
6  $CLOSED.initializeEmpty()$ ;  
7  $OPEN.insert(x_s)$ ;  
8 while not  $OPEN.empty()$  do  
9    $x \leftarrow OPEN.remove()$ ;  
10   $CLOSED.insert(x)$ ;  
11  foreach  $x' \in V$  such that  $(x, x') \in E$  do  
12    if  $x'.g = \infty$  then  
13       $x'.g \leftarrow x.g + c(x, x')$ ;  
14       $x'.parent \leftarrow x$ ;  
15       $OPEN.insert(x')$ ;  
16    else if  $x.g + c(x, x') \leq x'.g$  then  
17       $x'.g \leftarrow x.g + c(x, x')$ ;  
18       $x'.parent \leftarrow x$ ;  
19       $OPEN.rebalance(x')$ ;
```

Algorithm 3: Single source shortest path algorithm



Single Source Shortest Path Tree

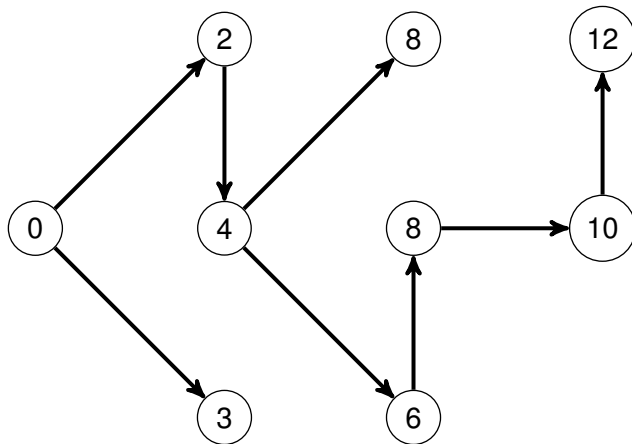


Figure: Tree produced by Single Source Shortest Path algorithm.



- The most fundamental algorithm in search based AI
- **informed** method: assumes the availability of an heuristic telling us *how good* a vertex is
 - *how good* is an estimate of the cost to go h from a vertex to the goal

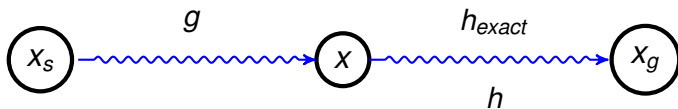


Figure: A node x along one path from x_s to x_g . The cost to come g is the cost of the best path (discovered so far) between x_s and x . h_{exact} is the cost of the shortest path between x and x_g , whereas h is an estimate of h_{exact} .



Important Quantities

- g : lowest cost-to-come discovered so far
- h_{exact} : lowest cost to go (unknown)
- $g + h_{exact}$: cost of the shortest path from x_s to x_g passing through x (after x is extracted from OPEN)
- h : estimate for h_{exact} (underestimation – more details later)
- $f = g + h$: is always an estimation of the cost of the shortest path from x_s to x_g passing through x



Admissible Heuristics

Definition

Let $G = (V, E)$ be a weighted graph where $c : V \rightarrow \mathbb{R}_{\geq 0}$ is the cost function. Let $x_g \in V$ be a goal vertex and for each vertex $v \in V$ let $c(p_{v,x_g})$ be the cost of a shortest path from v to x_g . An **admissible heuristic** is a function $h : V \rightarrow \mathbb{R}_{\geq 0}$ such that for each vertex $h(v) \leq c(p_{v,x_g})$.

- $h(v) = 0$ for each $v \in V$ is always an admissible heuristic because $c(p_{v,x_g}) \geq 0$ for each vertex v
- Dijkstra's algorithm can be seen as A^* with $h(v) = 0$ for each vertex
- "ideal heuristic": $h(v) = c(p_{v,x_g})$. The closer the gap, the better.



Consistent Heuristic

Definition

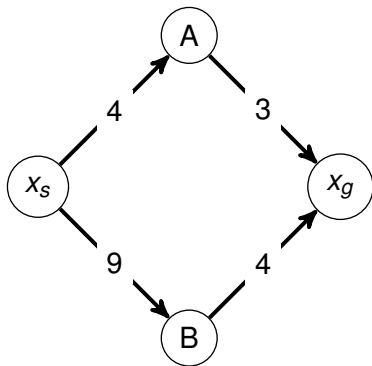
Let $G = (V, E)$ be a weighted graph where $c : V \rightarrow \mathbb{R}_{\geq 0}$ is the cost function, and let $x_g \in V$ be a goal vertex. A function $h : V \rightarrow \mathbb{R}_{\geq 0}$ is a **consistent heuristic** if

- ① for couple of vertices $u, v \in V$ with $(v, u) \in E$, then $h(v) \leq c(v, u) + h(u)$;
- ② $h(x_g) = 0$.

- does not over-estimate the cost of an edge
- consistent implies admissible, but admissible does not imply consistent



On heuristics: Admissible but not Consistent



Node	h
x_s	6
x_g	0
A	1
B	1

Table: Inconsistent estimate for the graph

Figure: A weighted directed graph where every edge is associated with a non-negative cost.



Consistent vs Admissible

- admissible is a requirement for correctness
- a consistent heuristic is more efficient (i.e., the algorithm expands less nodes)
- if admissible but not consistent, A* implementation is more complex



A* with Consistent Heuristic

- must keep two attributes per node: g and $f = g + h$
 - g is always the minimum cost of the shortest path discovered so far
- OPEN is prioritized by f
- everything else is like in Dijkstra's algorithm
- if $h(v) = 0$ for every vertex, A* behaves exactly like Dijkstra's algorithm



A* with Consistent Heuristic

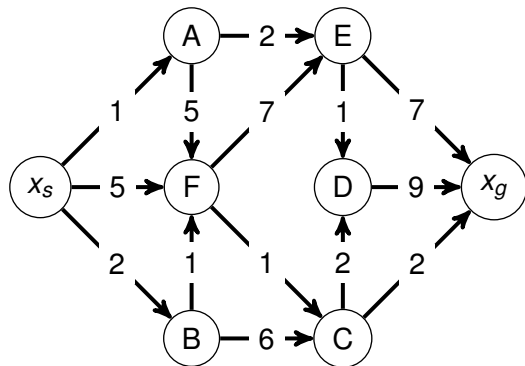
Data: $G = (V, E)$, $x_s \in V$, $x_g \in V$, $c : E \rightarrow \mathbb{R}_{\geq 0}$

Result: Shortest path from x_s to x_g if it exists, or FAILURE

```
1 foreach  $x \in V$  do
2   |  $x.parent \leftarrow \text{null}$ ;  $x.g \leftarrow \infty$ ;  $x.f \leftarrow \infty$ ;
3  $x_s.g \leftarrow 0$ ;  $x_s.f \leftarrow x_s.g + h(x_s)$ ;
4 OPEN.initializeEmpty();
5 CLOSED.initializeEmpty();
6 OPEN.insert( $x_s$ );
7 while not OPEN.empty() do
8    $x \leftarrow \text{OPEN.remove}()$ ;
9   CLOSED.insert( $x$ );
10  if  $x = x_g$  then
11    | return ExtractPath( $x_s, x_g$ );
12  foreach  $x' \in V$  such that  $(x, x') \in E$  do
13    if  $x'.g = \infty$  then
14      |  $x'.g \leftarrow x.g + c(x, x')$ ;
15      |  $x'.f \leftarrow x'.g + h(x')$ ;
16      |  $x'.parent \leftarrow x$ ;
17      | OPEN.insert( $x'$ );
18    else if  $x.g + c(x, x') \leq x'.g$  then
19      |  $x'.g \leftarrow x.g + c(x, x')$ ;
20      |  $x'.f \leftarrow x'.g + h(x')$ ;
21      |  $x'.parent \leftarrow x$ ;
22      | OPEN.rebalance( $x'$ );
23 return FAILURE;
```



A* Example



Node	h
x_s	5
x_g	0
A	6
B	3
C	1
D	4
E	4
F	2

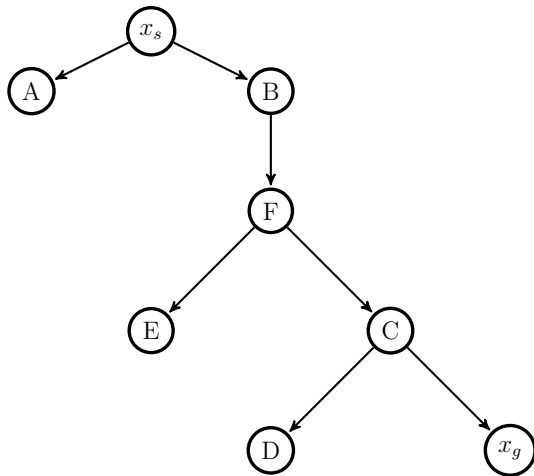


A* Example

OPEN	x_s	A	B	C	D	E	F	x_g
x_s	$N/0/5$	$N/\infty/\infty$	$N/\infty/\infty$	$N/\infty/\infty$	$N/\infty/\infty$	$N/\infty/\infty$	$N/\infty/\infty$	$N/\infty/\infty$
B, A, F	$N/0/5$	$x_s/1/7$	$x_s/2/5$	$N/\infty/\infty$	$N/\infty/\infty$	$N/\infty/\infty$	$x_s/5/7$	$N/\infty/\infty$
F, A, C	$N/0/5$	$x_s/1/7$	$x_s/2/5$	B/8/9	$N/\infty/\infty$	$N/\infty/\infty$	B/3/5	$N/\infty/\infty$
C, A, E	$N/0/5$	$x_s/1/7$	$x_s/2/5$	F/4/5	$N/\infty/\infty$	F/10/14	B/3/5	$N/\infty/\infty$
x_g , A, D, E	$N/0/5$	$x_s/1/7$	$x_s/2/5$	F/4/5	C/6/10	F/10/14	B/3/5	C/6/6

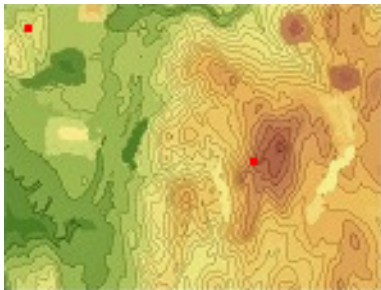


A* Spanning Tree

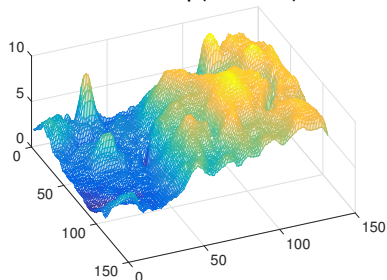


Comparing Different Planners

Elevation map (top view)



Elevation map (lateral view)



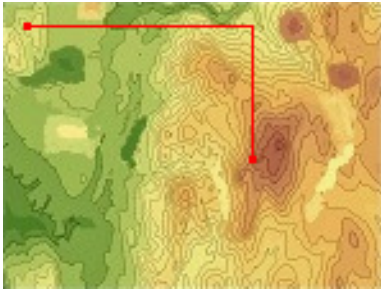
Elevation of a pixel $e(x)$:

Edge cost: downhill: 1; uphill $c(x, x') = 1 + K(e(x') - e(x))$



Comparing Different Planners: BFS

Path found by BFS

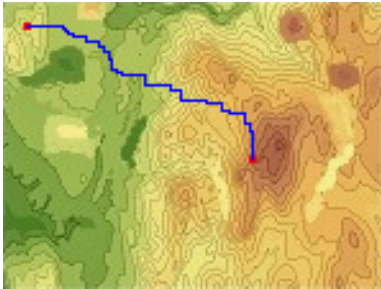


Nodes expanded by BFS



Comparing Different Planners: Dijkstra

Path found by Dijkstra

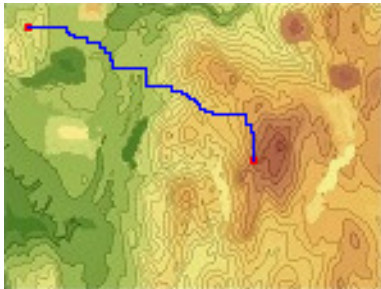


Nodes expanded by Dijkstra



Comparing Different Planners: A*

Path found by A*



Nodes expanded by A*



Navigation Functions

- encode a family of plans
 - for each state identifies the action execute
- intuition: for each state, define a non-negative function: $\psi : V \rightarrow \mathbb{R}_{\geq 0}$
- do gradient descent on ψ : among all edges outgoing from x , pick

$$e' = \arg \min_{(x,y) \in e(x)} \{\psi(y)\}$$

- pervasively used for obstacle avoidance (often in combination with other planners)



Navigation Function

Definition

Let $G = (V, E)$ be a planning graph and let $x_g \in V$ a goal vertex. A navigation function $\psi : V \rightarrow \mathbb{R}_{\geq 0}$ is *feasible* if it satisfies the following three conditions.

- 1 $\psi(x_g) = 0$.
- 2 If $x \in V$ is a vertex from which there is no path to x_g , then $\psi(x) = \infty$.
- 3 If x is a state from which x_g can be reached, then for $y = e'(x)$ we have $\psi(y) < \psi(x)$.



Navigation Functions

6	5	4	5	6	5
5	4	3	[Obstacle]		4
4	3	2			3
3	2	1	0	1	2
4	[Obstacle]		1	2	3
5			2	3	4

Figure: Navigation function for a grid world



Perception

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced

<https://sites.ucmerced.edu/scarpin>

<https://robotics.ucmerced.edu>



Why sensors?

- robots do not have all information available upfront and must acquire it on the go
- outcome of actions is not predictable; perception can assess deviations from desired outcomes
- time invariant, discrete, error-free sensor abstractions

$$\mathbf{z}_t = h(\mathbf{x}_t, \mathbf{u}_t) \qquad \mathbf{z}_t = h(\mathbf{x}_t)$$

- sensor reading at time t is a function of the state $\mathbf{x}$ at time t and implicitly of the environment.
- if the environment is static (an assumption we will often make), then it can be modeled into h



Sensor models

- sensors are noisy, too
- different models for sensor readings

$$\mathbf{z}_t = h(\mathbf{x}_t, \mathbf{u}_t, \psi_t)$$

- additive noise model; h_c is the *correct* sensor reading that we cannot access

$$h(\mathbf{x}_t, \mathbf{u}_t, \psi_t) = h_c(\mathbf{x}_t, \mathbf{u}_t) + \psi_t$$

- In estimation theory we will study how to handle these error sources; here we focus on the ROS API to retrieve sensor reading values.



Dead Reckoning

- class of sensors producing an estimate at time t by combining initial condition with temporal integration

$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \dot{\mathbf{x}}(\nu) d\nu$$

- not strictly adherent to the model we saw before
- initial condition must be given (often arbitrarily set)
- error grows over time
- typical example: wheel encoders



Types of Sensors

Proprioceptive return information about the robot itself

- encoders, accelerometers, gyroscopes, inertial measurement units, etc.

Exteroceptive return information about the environment

- sonars, laser range finders, 3D scanners/3D cameras, cameras, GPS, contact sensors



Sensors in ROS

- most sensors come with ROS nodes to interact with them (*drivers*)
- information coming from a sensor is made available through topics
- your code typically does not interact with the sensor (the driver does), but rather gets its information from the topic)
- messages sent through sensor related topics are part of the `sensor_msgs` package



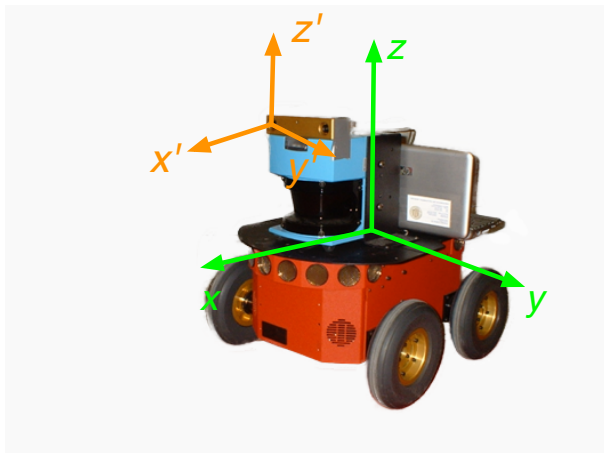
Sensors in ROS

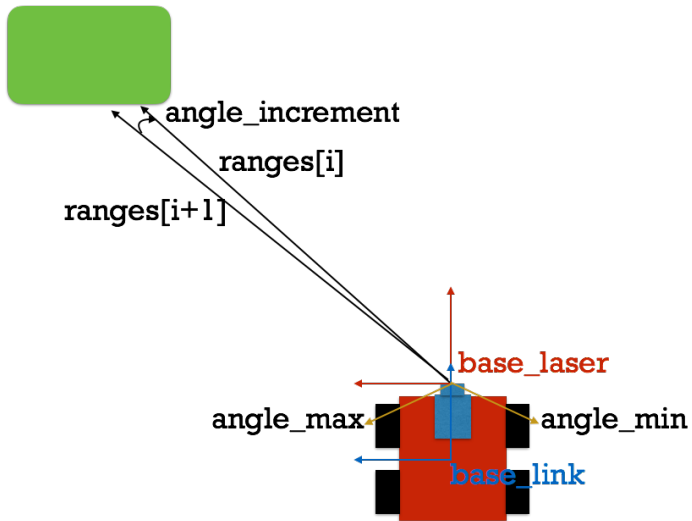
Message Type	Sensor
BatteryState	Status of battery (e.g., voltage and other info)
Image	Camera image
Imu	Inertial Measurement Unit
JointState	Status of set of joints (position, velocity, torque)
LaserScan	Planar range finder (e.g., laser)
NavSatFix	GPS
PointCloud	Collection of 3D points returned by a 3D scanner
PointCloud2	Collection of 3D points returned by a 3D scanner (2nd version)
Range	Single range reading (e.g., sonar)

Exception: `nav_msgs::msg::Odometry`



LaserScan





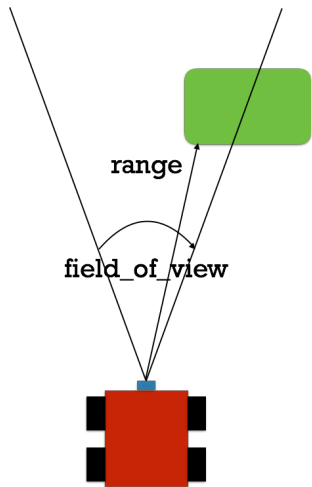
sensor_msgs::msg::LaserScan

```
std_msgs/Header header  
float32  angle_min  
float32  angle_max  
float32  angle_increment  
float32  time_increment  
float32  scan_time  
float32  range_min  
float32  range_max  
float32[] ranges  
float32[] intensities
```

Code example: pubsubstl.cpp



Sonar



sensor_msgs::msg::Range

```
uint8 ULTRASOUND=0
uint8 INFRARED=1
std_msgs/Header header
uint8 radiation_type
float32 field_of_view
float32 min_range
float32 max_range
float32 range
float variance
```



- wide variety of sensors in this category
- `sensor_msgs::msg::Imu` aims at including fields for most IMUs; not all may be used in practice
- covariance information included to assess precision



sensor_msgs::Imu

Header header

geometry_msgs/Quaternion orientation
float64[9] orientation_covariance

geometry_msgs/Vector3 angular_velocity
float64[9] angular_velocity_covariance

geometry_msgs/Vector3 linear_acceleration
float64[9] linear_acceleration_covariance



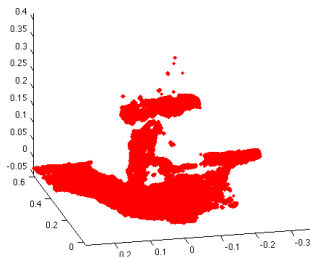
GPS: sensor_msgs::msg::NavSatFix

```
std_msgs/Header header
NavSatStatus status
float64 latitude
float64 longitude
float64 altitude
float64[9] position_covariance
uint8 COVARIANCE_TYPE_UNKNOWN=0
uint8 COVARIANCE_TYPE_APPROXIMATED=1
uint8 COVARIANCE_TYPE_DIAGONAL_KNOWN=2
uint8 COVARIANCE_TYPE_KNOWN=3
uint8 position_covariance_type
```



PointCloud

- two versions (PointCloud and PointCloud2)
- can be produced by a variety of sensors (e.g., Velodyne, RealSense, ...)
- ROS can be interfaced with external libraries (e.g., PCL) to utilize read-to-use algorithms



PointCloud

```
std_msgs/Header header
geometry_msgs/Point32[] points
  float32 x
  float32 y
  float32 z
sensor_msgs/ChannelFloat32[] channels
  string name
  float32[] values
```



PointCloud2

```
std_msgs/Header header
uint32 height
uint32 width
PointField[] fields
bool is_bigendian
uint32 point_step  # Length of a point in bytes
uint32 row_step    # Length of a row in bytes
uint8[] data       # Actual point data, size is (row_step*height)
bool is_dense      # True if there are no invalid points
```



Odometry

- derived from robot geometry and commands given to the robot
- provided by the navigation stack; hence via the `nav_msgs` package



nav_msgs::msg::Odometry

std_msgs/Header header

Frame id the pose points to. The twist is in this coordinate frame.
string child_frame_id

Estimated pose that is typically relative to a fixed world frame.
geometry_msgs/PoseWithCovariance pose

Estimated linear and angular velocity relative to child_frame_id.
geometry_msgs/TwistWithCovariance twist



Estimation and Filtering

Stefano Carpin

Department of Computer Science and Engineering
School of Engineering
University of California, Merced

<https://sites.ucmerced.edu/scarpin>

<https://robotics.ucmerced.edu>



Estimation problem

- the process of determining a quantity from measurements that are *incomplete* and/or *inaccurate*.
- quantity being estimated can be a scalar or a vector
- Example: determine latitude and longitude from a set of readings coming from satellites (GPS)
- estimation often involves multiple **heterogeneous** measurements
 - sensor fusion



Estimation problem in robotics

- should consider both sensor readings and actions
- sensor readings decrease uncertainty, actions increase uncertainty
- classic estimation problem in robotics: localization
 - where is the robot? (e.g., x, y, ϑ)

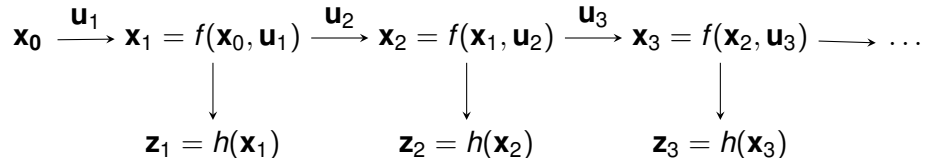


Filtering

- an estimation problem where the quantity being estimated changes overtime
 - E.g., the state of a dynamical system, like the pose of a robot
 - more complicated than classic estimation
- solution must be updated over time
- algorithms solving this type of problems are typically called **filters**



Estimation problem in robotics

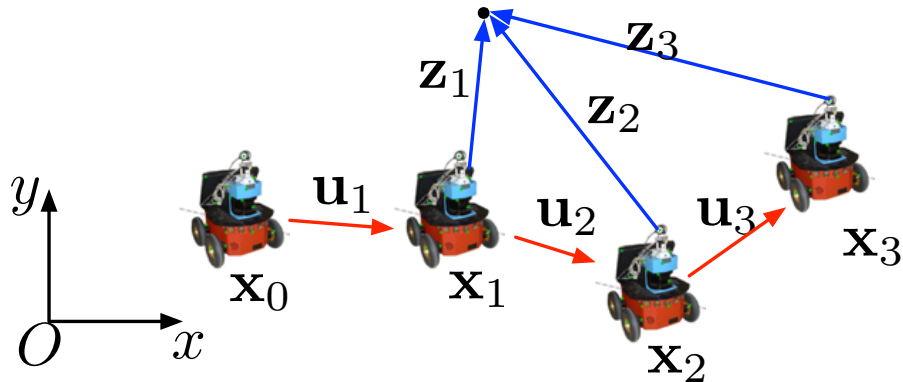


Estimation problem in robotics

$$\begin{array}{ccccccc} \mathbf{x}_0 & \xrightarrow{\mathbf{u}_1} & \mathbf{x}_1 = f(\mathbf{x}_0, \mathbf{u}_1, \zeta_1) & \xrightarrow{\mathbf{u}_2} & \mathbf{x}_2 = f(\mathbf{x}_1, \mathbf{u}_2, \zeta_1) & \xrightarrow{\mathbf{u}_3} & \mathbf{x}_3 = f(\mathbf{x}_2, \mathbf{u}_3, \zeta_3) \longrightarrow \dots \\ & & \downarrow & & \downarrow & & \downarrow \\ & & \mathbf{z}_1 = h(\mathbf{x}_1, \psi_1) & & \mathbf{z}_2 = h(\mathbf{x}_2, \psi_2) & & \mathbf{z}_3 = h(\mathbf{x}_3, \psi_3) \end{array}$$



Estimation problem in robotics



- We seek an algorithm computing the function

$$\hat{\mathbf{x}}_t = g(\mathbf{x}_0, \mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_t, \mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_t) \quad (1)$$

- $\hat{\mathbf{x}}_t$ is the estimate for $\mathbf{x}$ at time t .
- Note the dependency on time because $\mathbf{x}$ changes over time, and so does the estimate.



Classical Vs Bayesian Approach to Estimation

- **Classical Approach**: quantity being estimated is an unknown constant
- **Bayesian Approach**: quantity being estimated is assumed to be random variable from a set Θ of possible values
 - a prior over Θ is assumed, and the objective is to produce a posterior

We will embrace the Bayesian method, i.e., our objective is to produce a posterior over Θ



Estimation error

- Difference between our estimate and the quantity being estimated:

$$\mathbf{e}_t = \hat{\mathbf{x}}_t - \mathbf{x}_t. \quad (2)$$

- both $\mathbf{x}$ and $\hat{\mathbf{x}}$ are random variables and so is the error.
- Estimate error should be *small*.
- Ideally we want to minimize this error metric:

$$\mathbb{E}_{\mathbf{x}_t, \hat{\mathbf{x}}_t}[\|\mathbf{e}_t\|^2] = \mathbb{E}_{\mathbf{x}_t, \hat{\mathbf{x}}_t}[\|\hat{\mathbf{x}}_t - \mathbf{x}_t\|^2] \quad (3)$$

- However, $\mathbf{x}_t$ is not known...
 - intuition: minimize difference between sensor readings (known) and *predicted* sensor reading (that can be computed)



Recursive estimation

Eq. (1) has a potential problem:

- As more and more data is collected over time, the estimation algorithm would depend on more and more inputs, and this could slow it down if they have to be all taken into account

Recursive estimation:

$$\hat{\mathbf{x}}_t = g(\hat{\mathbf{x}}_{t-1}, \mathbf{u}_t, \mathbf{z}_t). \quad (4)$$

New estimate depends just on previous estimate and latest input and sensor reading.



Parametric Vs Non-Parametric Estimation

In Bayesian estimation $\mathbf{x}_t$ is a random variable. The estimation algorithm g computes a posterior PDF $\hat{\mathbf{x}}$.

- **Parametric Estimation:** $\hat{\mathbf{x}}$ is a random variable with a PDF determined by a finite number of parameters (e.g., Gaussian). Estimation algorithm determines the parameters (e.g., Kalman Filter)
- **Non-parametric Estimation:** $\hat{\mathbf{x}}$ is a random variable whose PDF is not described in closed form by a finite set of parameters. Estimation algorithm produces an approximation (e.g., particle filter)



Discrete estimation algorithms

Objective: estimate PMF of a variable with a finite alphabet $x_1, \dots, x_n$, i.e., we want:

$$P(X = x_1) \quad P(X = x_2) \quad \dots \quad P(X = x_n)$$

Example: we may discretize the set of possible locations of a robot and we want to assign a probability to each location.



Bayes estimation

All algorithms we will see rely on Bayes rule:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}.$$

Recall its variants, in particular those with background knowledge (i.e., conditioning on multiple events.) We assume to know $P(B|A)$ and $P(A)$, but we will not need $P(B)$.



Bayes rule in action

$$P(X = x_1|Z = z) = \frac{P(Z = z|X = x_1)P(X = x_1)}{P(Z = z)}$$

$$P(X = x_2|Z = z) = \frac{P(Z = z|X = x_2)P(X = x_2)}{P(Z = z)}$$

...

$$P(X = x_n|Z = z) = \frac{P(Z = z|X = x_n)P(X = x_n)}{P(Z = z)}$$

Denominator $P(Z = z)$ is the same in all expressions, and $\sum_{i=1}^n P(X = x_i|Z = z) = 1$.



Bayes rule in action

$$\begin{aligned}\sum_{i=1}^n P(X = x_i | Z = z) &= \sum_{i=1}^n \frac{P(Z = z | X = x_i) P(X = x_i)}{P(Z = z)} \\ &= \frac{1}{P(Z = z)} \sum_{i=1}^n P(Z = z | X = x_i) P(X = x_i) = 1\end{aligned}$$

Hence:

$$P(Z = z) = \sum_{i=1}^n P(Z = z | X = x_i) P(X = x_i)$$

Hence we write

$$P(x_i | z) = \eta P(z | x_i) P(x_i)$$

because the normalization factor η is a constant that can be computed separately and can be assumed known.



Integrating multiple sensor readings

Assume we now have two sensor readings z_1, z_2 . We assume they are **independent**, i.e., $P(z_1|z_2) = P(z_1)$ and $P(z_2|z_1) = P(z_2)$.

$$P(x_i|z_1, z_2) = \frac{P(z_1|x_i, z_2)P(x_i|z_2)}{P(z_1|z_2)} = \frac{P(z_1|x_i)P(x_i|z_2)}{P(z_1)}.$$

Apply Bayes rule again to $P(x_i|z_2)$:

$$P(x_i|z_1, z_2) = \frac{P(z_1|x_i)P(x_i|z_2)}{P(z_1)} = \frac{P(z_1|x_i)}{P(z_1)} \frac{P(z_2|x_i)P(x_i)}{P(z_2)} = \frac{P(z_1|x_i)}{P(z_1)} \frac{P(z_2|x_i)}{P(z_2)} P(x_i)$$



Estimation with multiple readings

$$\begin{aligned} P(x_i | z_1, z_2, \dots, z_n) &= \frac{P(z_1 | x_i)}{P(z_1)} \frac{P(z_2 | x_i)}{P(z_2)} \dots \frac{P(z_n | x_i)}{P(z_n)} P(x_i) \\ &= \left(\prod_{j=1}^n \frac{P(z_j | x_i)}{P(z_j)} \right) P(x_i) = \eta \left(\prod_{j=1}^n P(z_j | x_i) \right) P(x_i) \end{aligned} \quad (5)$$

Each sensor reading corresponds to a **scaling** of the prior. Therefore **the order in which they are considered does not matter**.



Example: localization

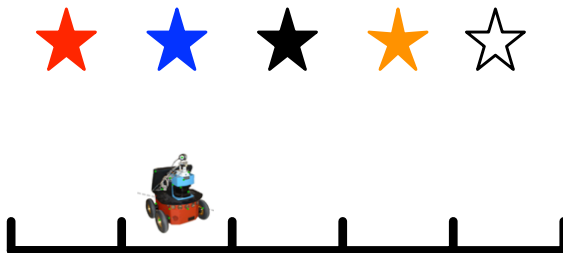


Figure: Example 6.4 in the lecture notes

Sensor readings: black, red, black, black, black
Nonuniform prior



Posterior propagation over time

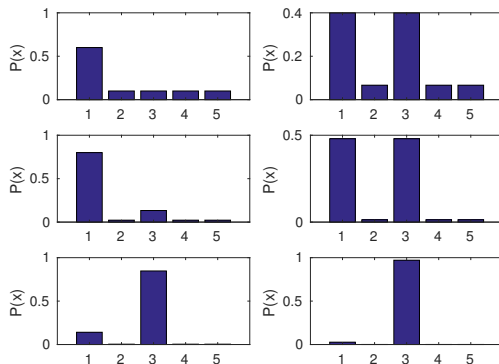


Figure: Prior (top left) and posterior (remaining diagrams) after the various sensor readings are sequentially considered (see lecture notes for numerical details).



Sensor model and motion model

- $P(z|x)$ is the probabilistic version of the state observation equation $z = h(x)$. This is called **sensor model**.
- Similarly we can consider the probabilistic version of the state transition equation $x_t = f(x_{t-1}, u_t)$. This will be $P(x'|x, u)$ and it will be called **motion model**.



Estimation with inputs

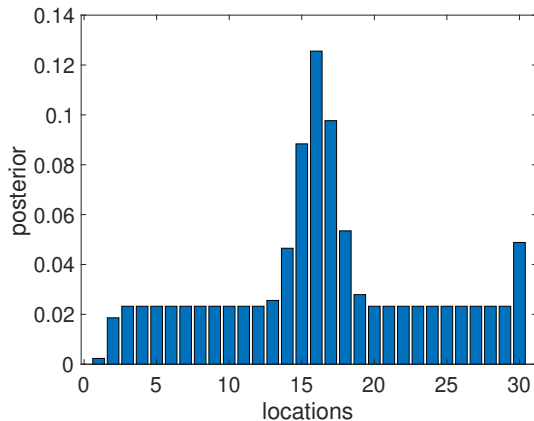
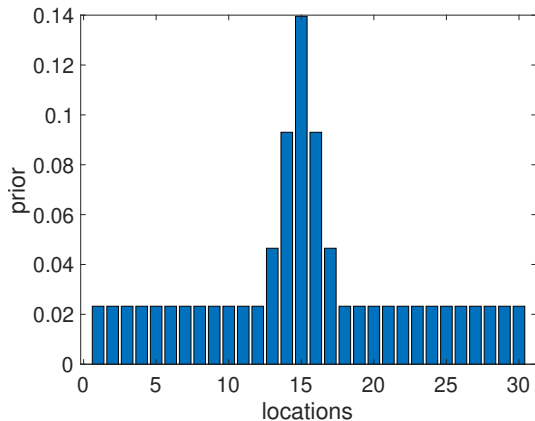
$$P(x|u) = \sum_{i=1}^n P(x|y_i, u)P(y_i).$$

Where we used the total probability theorem.

Note that we assume to know the robot motion model and the prior.



Uncertainty Propagation



Estimation with multiple inputs

$$P(x|u_1, u_2) = \sum_{i=1}^n P(x|y_i, u_1, u_2)P(y_i|u_1).$$

Assume the motion model has the *Markov* property, i.e., the next state depends only on the current state and current input, but not the past. Therefore:

$$P(x|u_1, u_2) = \sum_{i=1}^n P(x|y_i, u_2)P(y_i|u_1).$$

Further expand last term:

$$P(x|u_1, u_2) = \sum_{i=1}^n \left[P(x|y_i, u_2) \sum_{j=1}^n P(y_i|y_j, u_1)P(y_j) \right].$$



Estimation with multiple inputs

If we now swap the order of the inputs we get the following expression:

$$P(x|u_2, u_1) = \sum_{i=1}^n \left[P(x|y_i, u_1) \sum_{j=1}^n P(y_i|y_j, u_2) P(y_j) \right]$$

Therefore, for the case of inputs *the order matters*.



Recursive Discrete Bayes Filter

Objective: determine

$$P(x_t | x_0, u_1, \dots, u_t, z_1, \dots, z_t).$$

Recall the formulation of our estimation problem:

$$\hat{x}_t = P(x_t | x_0, u_1, \dots, u_t, z_1, \dots, z_t).$$



Discrete Bayes Filter

$$\begin{aligned}\hat{x}_t &= P(x_t | x_0, u_1, \dots, u_t, z_1, \dots, z_t) \\ &= \frac{P(z_t | x_0, u_1, \dots, u_t, z_1, \dots, z_{t-1}, x_t) P(x_t | x_0, u_1, \dots, u_t, z_1, \dots, z_{t-1})}{P(z_t | x_0, u_1, \dots, u_t, z_1, \dots, z_{t-1})} \\ &= \frac{P(z_t | x_t) P(x_t | x_0, u_1, \dots, u_t, z_1, \dots, z_{t-1})}{P(z_t)} \\ &= \frac{P(z_t | x_t) \sum_{x_{t-1}} P(x_t | x_0, u_1, \dots, u_t, z_1, \dots, z_{t-1}, x_{t-1}) P(x_{t-1} | x_0, u_1, \dots, u_t, z_1, \dots, z_{t-1})}{P(z_t)}\end{aligned}$$



Discrete Bayes Filter

$$\begin{aligned} &= \frac{P(z_t|x_t) \sum_{x_{t-1}} P(x_t|x_{t-1}, u_t) P(x_{t-1}|x_0, u_1, \dots, u_t, z_1, \dots, z_{t-1})}{P(z_t)} \\ &= \frac{P(z_t|x_t) \sum_{x_{t-1}} P(x_t|x_{t-1}, u_t) P(x_{t-1}|x_0, u_1, \dots, u_{t-1}, z_1, \dots, z_{t-1})}{P(z_t)} \\ &= \frac{P(z_t|x_t) \sum_{x_{t-1}} P(x_t|x_{t-1}, u_t) \hat{x}_{t-1}}{P(z_t)} \\ &= \eta P(z_t|x_t) \sum_{x_{t-1}} P(x_t|x_{t-1}, u_t) \hat{x}_{t-1} \end{aligned}$$



Discrete Bayes Filter

- $\sum_{x_{t-1}} P(x_t | x_{t-1}, u_t) \hat{x}_{t-1}$ is the **prediction** step and corresponds to an increase in uncertainty.
- $\eta P(z_t | x_t)$ is the **correction** step and corresponds to a decrease in uncertainty.
- prediction and correction are two pervasive concepts in all filtering algorithms we will see



Discrete Bayes Filter Algorithm

Data: $\hat{x}_{t-1}, z_t, u_t$

Result: Estimate $\hat{x}_t$

```
1 for  $i \leftarrow 1$  to  $n$  do  
2   |  $x'_i \leftarrow \sum_{x_{t-1}} P(x'_i | x_{t-1}, u_t) \hat{x}_{t-1};$   
3 for  $i \leftarrow 1$  to  $n$  do  
4   |  $x''_i \leftarrow P(z_t | x_i) x'_i;$   
5  $\frac{1}{\eta} \leftarrow \sum_{i=1}^n x''_i;$   
6 for  $i \leftarrow 1$  to  $n$  do  
7   |  $\hat{x}_i \leftarrow \eta x''_i;$ 
```

Algorithm 1: Discrete Bayes Filter



Probabilistic Motion Models

Data: $x_{t+1} = (x', y', \vartheta')$, $x_t = (x, y, \vartheta)$, $u_t = (v_t, v_r)$

Result: $P(x_{t+1}|x_t, u_t)$

```
1  $u' = (v'_t, v'_r) \leftarrow \text{InverseModel}(x_{t+1}, x_t);$   
2  $\delta\vartheta \leftarrow \frac{\vartheta' - \vartheta}{\Delta t} - v'_r;$   
3 return  $f_G(v_t - v'_t, \alpha_1 v_t^2 + \alpha_2 v_r^2) \cdot f_G(v_r - v'_r, \alpha_3 v_t^2 + \alpha_4 v_r^2) \cdot f_G(\delta\vartheta, \alpha_5 v_t^2 + \alpha_6 v_r^2);$ 
```

Algorithm 2: Probabilistic Motion Model



Discrete Bayes Filter Tradeoffs

- performance quadratic in the number of states
- non-parametric algorithm, so does not make any assumption about distribution being estimated
- typical exponential dependency between the dimensions of the state space and the number of states
 - can be managed for a mobile robot (three dimensional state space), but becomes quickly problematic (*curse of dimensionality*)
- computes estimates everywhere, even in regions with low probability
- typically uses uniform resolution even when inappropriate
 - adaptive resolution possible, but complicates the implementation

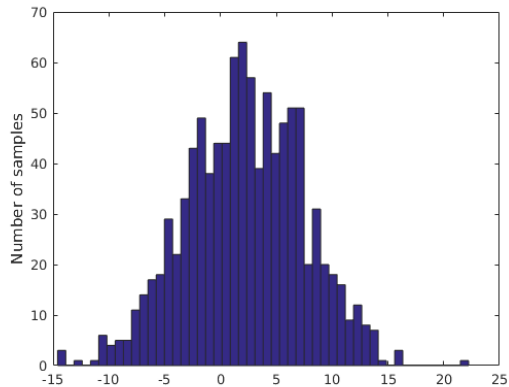


Particle Filters

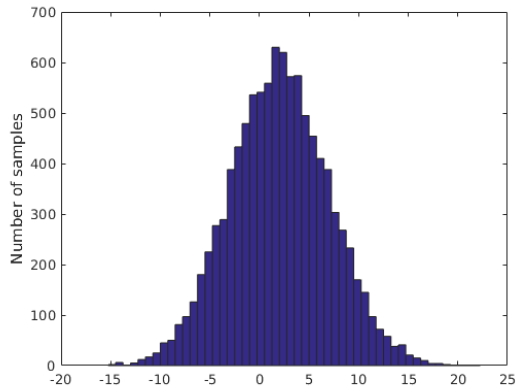
- ideal for non-parametric (e.g., non Gaussian), non linear estimation problems
- idea: represent the posterior through a set of samples (called **particles**) drawn from it
 - more samples where the PDF is high, less samples where the PDF is small
- figuratively, one can think at every particle as a possible hypothesis for the solution to the estimation problem



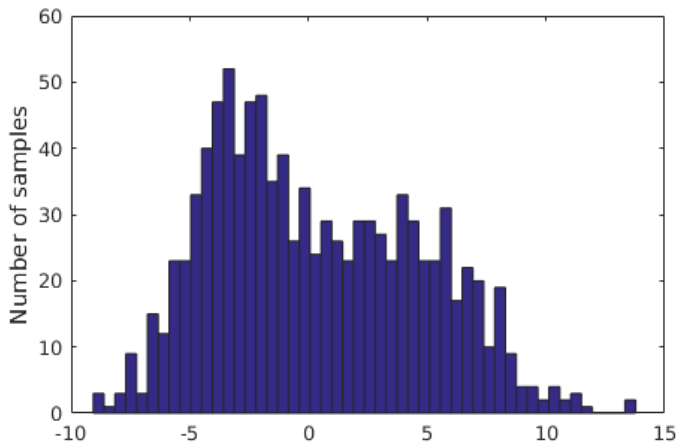
Sampling from posteriors



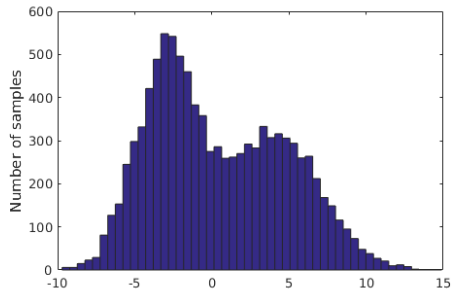
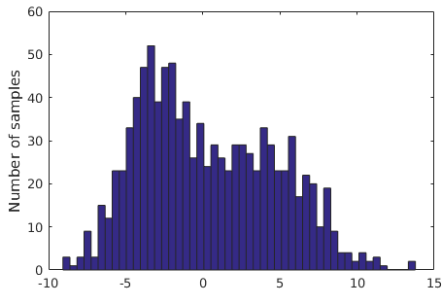
Sampling from posteriors



Sampling from posteriors



Sampling from posteriors



Challenges

- we can not sample from the posterior PDF
 - the very reason why we are solving an estimation problem is because we want to determine such PDF
- we can sample from other distributions we know
 - once again we need the *motion model* and *sensor model* – this time represented as PDFs



Particle filters

- posterior at time $t - 1$ is represented by particles set $\mathcal{P}_{t-1}$
- recursive estimation: given $\mathcal{P}_{t-1}$, z_t , and u_t , determine new particle set $\mathcal{P}_t$
- probabilistic motion model PDF: $f(x'|x, u)$
- probabilistic sensor model PDF: $g(h|x)$
- these PDFs are easy to determine and ready available
- each particle is assigned a weight, i.e., *how well it explains the data*.
- build next set of particles retaining particles with high weight
 - can be thought as *survival of the fittest*, but with much caution
- the probability of retaining a particle is proportional to its weight



Particle filters

- 1 Create new (intermediate) particles x_t^i , from old ones $\hat{x}_{t-1}$ (one each), sampling from the probabilistic motion model $f(x'|x, u)$
- 2 Assign each intermediate particle a weight based on the probabilistic sensor model $g(h|x)$
- 3 Sample new particles $\hat{x}_t$ from intermediate particles x_t^i



Particle Filter Algorithm

Data: $\mathcal{P}_{t-1}, z_t, u_t$

Result: New set of particles $\mathcal{P}_t$

```
1 for  $i \leftarrow 1$  to  $N$  do
2    $x_t^i \sim f(\hat{x}_{t-1}^i, u_t)$ ;
3    $w_i \leftarrow g(z_t | x_t^i)$ ;
4 for  $i \leftarrow 1$  to  $N$  do
5    $\hat{x}_t^i \sim$  sample from  $\mathcal{P}'_t$  with probability proportional to  $w_i$ ;
```



Sample Generation

Data: $x_t = (x, y, \vartheta)$, $u_t = (v_t, v_r)$

Result: $x_{t+1} \sim f(x_t, u_t)$

```
1  $v'_t \leftarrow v_t + s(\alpha_1 v_t^2 + \alpha_2 v_r^2);$   
2  $v'_r \leftarrow v_r + s(\alpha_3 v_t^2 + \alpha_4 v_r^2);$   
3  $\delta\vartheta \leftarrow s(\alpha_5 v_t^2 + \alpha_6 v_r^2);$   
4  $(x', y', \vartheta') \leftarrow \text{ForwardModel}(x, y, \vartheta, v'_t, v'_r);$   
5  $\vartheta' \leftarrow \vartheta' + \delta\vartheta \Delta t;$   
6 return  $(x', t', \vartheta');$ 
```

Algorithm 3: Sample generation

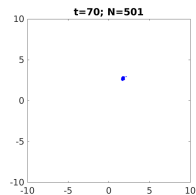
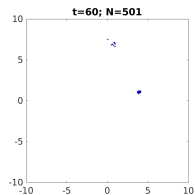
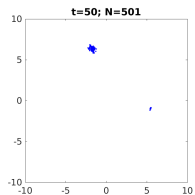
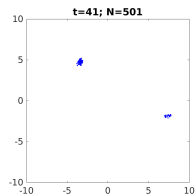
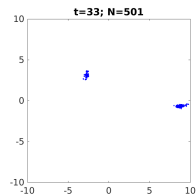
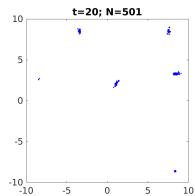
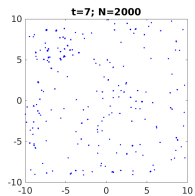
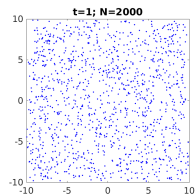


Implementation details

- how many particles?
 - not an easy answer. Answer is problem-dependent and requires experience
 - the number of particles is not necessarily constant but may vary
 - good reasons to have many particles and few particles
- how do we initialize particles set?
 - draw initial set from prior (e.g., Gaussian)
 - if no prior given, assume prior is a uniform (represents lack of knowledge)



Example



Kalman Filter

- parametric estimation technique for the case where posterior is represented by a multivariate Gaussian
 - algorithm determines mean vector and covariance matrix
- recursive estimation technique
- **optimal** under Gaussian and linear hypotheses
- **linear** refers to the state transition equation and the observation equations
- can be extended to the nonlinear case (but then no-longer optimal)



Relevant facts

- Let $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_x, \boldsymbol{\Sigma}_x)$. Then

$$\mathbf{y} = \mathbf{A}\mathbf{x}.$$

is also a Gaussian random vector $\boldsymbol{\mu}_y \sim \mathcal{N}(\mathbf{A}\boldsymbol{\mu}_x, \mathbf{A}\boldsymbol{\Sigma}_x\mathbf{A}^T)$

- in general $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$ means $\boldsymbol{\mu}_y \sim \mathcal{N}(\mathbf{A}\boldsymbol{\mu}_x + \mathbf{b}, \mathbf{A}\boldsymbol{\Sigma}_x\mathbf{A}^T)$
- $\mathbf{x}$ is the sum of two independent Gaussian vectors $\mathbf{y}$ and $\mathbf{z}$

$$\mathbf{x} = \mathbf{y} + \mathbf{z}$$

then $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_y + \boldsymbol{\mu}_z, \boldsymbol{\Sigma}_y + \boldsymbol{\Sigma}_z)$



Relevant facts

- if $\mathbf{x}$ is a random vector (not necessarily Gaussian) and $\mathbf{y} = \mathbf{Ax} + \mathbf{b}$
- then $\boldsymbol{\mu}_y = \mathbf{A}\boldsymbol{\mu}_x + \mathbf{b}$ and $\boldsymbol{\Sigma}_y = \mathbf{A}\boldsymbol{\Sigma}_x\mathbf{A}^T$
 - easy to show from definitions of mean and variance
- however mean and variance in this case do not necessarily fully describe $\mathbf{y}$'s PDF



Relevant case

- Let $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_x, \boldsymbol{\Sigma}_x)$ and $\mathbf{y} \sim \mathcal{N}(\boldsymbol{\mu}_y, \boldsymbol{\Sigma}_y)$ be two **independent** random vectors.
- Let $\mathbf{b}$ be a constant vector.
- Let $\mathbf{z} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} + \mathbf{b}$.
- then $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}_z, \boldsymbol{\Sigma}_z)$
- with $\boldsymbol{\mu}_z = \mathbf{A}\boldsymbol{\mu}_x + \mathbf{B}\boldsymbol{\mu}_y + \mathbf{b}$ and $\boldsymbol{\Sigma}_z = \mathbf{A}\boldsymbol{\Sigma}_x\mathbf{A}^T + \mathbf{B}\boldsymbol{\Sigma}_y\mathbf{B}^T$.



Linear systems

- State transition equation:

$$\mathbf{x}_t = \mathbf{A}_t \mathbf{x}_{t-1} + \mathbf{B}_t \mathbf{u}_t + \mathbf{v}_t$$

where $\mathbf{v}_t$ is a Gaussian vector with 0 mean and covariance matrix $\mathbf{R}_t$

- State observation equation

$$\mathbf{z}_t = \mathbf{H}_t \mathbf{x}_t + \mathbf{w}_t$$

where $\mathbf{w}_t$ is a Gaussian vector with 0 mean and covariance matrix $\mathbf{Q}_t$.

- additional independence assumptions
 - $\mathbf{v}_i$ is independent from $\mathbf{v}_j$ for each i, j
 - $\mathbf{w}_i$ is independent from $\mathbf{w}_j$ for each i, j
- $\mathbf{x}$ will be a random vector (even when $\mathbf{x}_0$ is deterministically known)



Relevant quantities for the Kalman Filter (KF)

- μ_{t-1} : mean of the estimate for $\mathbf{x}$ at time $t - 1$;
- Σ_{t-1} : covariance of the estimate for $\mathbf{x}$ at time $t - 1$;
- $\mathbf{u}_t$: input at time t ;
- $\mathbf{z}_t$: observation at time t ;
- $\mathbf{R}_t$: covariance matrix for the system evolution noise at time t ;
- $\mathbf{Q}_t$: covariance matrix for the observation noise at time t .

Recall that the objective is to estimate mean and covariance of $\mathbf{x}$ at time t



KF algorithm

Prediction:

- 1 $\bar{\mu}_t = \mathbf{A}_t \mu_{t-1} + \mathbf{B}_t \mathbf{u}_t$
- 2 $\bar{\Sigma}_t = \mathbf{A}_t \Sigma_{t-1} \mathbf{A}_t^T + \mathbf{R}_t$

Correction:

- 1 $\mathbf{K}_t = \bar{\Sigma}_t \mathbf{H}_t^T (\mathbf{H}_t \bar{\Sigma}_t \mathbf{H}_t^T + \mathbf{Q}_t)^{-1}$
- 2 $\mu_t = \bar{\mu}_t + \mathbf{K}_t (\mathbf{z}_t - \mathbf{H}_t \bar{\mu}_t)$
- 3 $\Sigma_t = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \bar{\Sigma}_t$

Note that in order to *bootstrap* the method, we need an initial estimate for the mean and covariance, i.e., we need μ_0 and Σ_0 .



KF Algorithm

- KF method provides an *optimal* estimate. This optimality is measured with respect to

$$\mathbb{E}_{\mathbf{x}_t, \hat{\mathbf{x}}_t} [||\mathbf{e}_t||^2] = \mathbb{E}_{\mathbf{x}_t, \hat{\mathbf{x}}_t} [||\hat{\mathbf{x}}_t - \mathbf{x}_t||^2]$$

- if the vector $\mathbf{u}_t$ is also a Gaussian vector with mean μ_{ut} and covariance matrix Σ_{ut} , then the first equation in the prediction step becomes $\bar{\mu}_t = \mathbf{A}_t \mu_{t-1} + \mathbf{B}_t \mu_{ut}$ and the second equation becomes $\bar{\Sigma}_t = \mathbf{A}_t \Sigma_{t-1} \mathbf{A}_t^T + \mathbf{B}_t \Sigma_{ut} \mathbf{B}_t^T + \mathbf{R}_t$. The correction step instead does not change.
- in a practical scenario it could be that inputs and observations do not alternate one to one, e.g., the system could receive an observation every three inputs or multiple observations in a row. In such case the step corresponding to the missing data is simply skipped, i.e., one would perform multiple predictions in sequence or multiple corrections in sequence.
- the term $(\mathbf{z}_t - \mathbf{H}_t \bar{\mu}_t)$ is called *innovation*.



Unidimensional example

Let us consider a unidimensional case governed by the following simple equations:

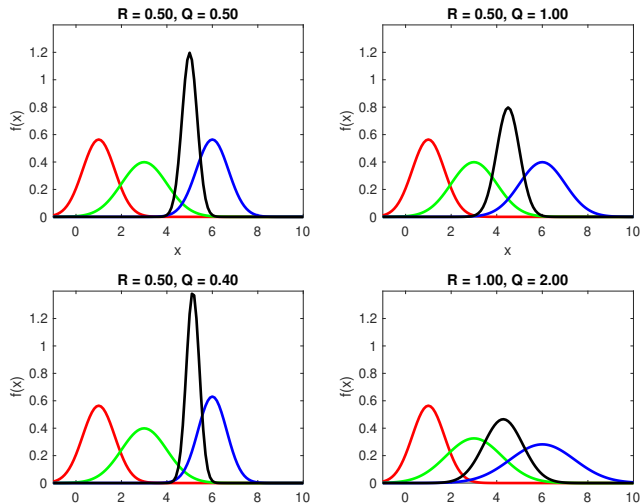
$$X_t = X_{t-1} + U_t$$

$$Z_t = X_t$$

Let v_t and w_t have constant covariance, as specified in the following. The first input u_1 is 2 and the first sensor reading z_1 is 6. Finally, let $x_0 \sim \mathcal{N}(1, 0.5)$.



Unidimensional example



A robot with two sensors

We now consider a small variant of the previous example, where the robot is now equipped with two sensors returning the value of the unidimensional state. This is modeled as follows:

$$x_t = x_{t-1} + u_t$$

$$z_t = \mathbf{H}x_t$$

$$\mathbf{H} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \mathbf{Q} = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.5 \end{bmatrix}$$

Let $x_0 \sim \mathcal{N}(1, 0.5)$, let $u_1 = 1$ and $Q = 1$. Moreover, let the first sensor reading be $\mathbf{z}_1 = [3 \quad 3]'$.



A robot with two sensors

Prediction: $\bar{\mu}_1 = 2$ and $\bar{\sigma}_1^2 = 1$

Correction:

$$\mathbf{K}_1 = \bar{\sigma}_1^2 \mathbf{H}^T (\mathbf{H} \bar{\sigma}_1^2 \mathbf{H}^T + \mathbf{Q})^{-1} = [0.5263 \quad 0.2105]$$

$$\mu_1 = \bar{\mu}_1 + \mathbf{K}_1 (\mathbf{z}_1 - \mathbf{H} \bar{\mu}_1) = 2.3158$$

$$\sigma_1 = (1 - \mathbf{K} \mathbf{H}) \bar{\sigma}_1^2 = 0.0526$$

Note that the Kalman gain gives more weight to the first sensor.



Extended Kalman Filter

In most cases, state transition equations and sensor functions are non-linear:

$$\mathbf{x}_t = f_t(\mathbf{x}_{t-1}, \mathbf{u}_t) + \mathbf{v}_t$$

$$\mathbf{z}_t = h_t(\mathbf{x}_t) + \mathbf{w}_t$$

In this case we can linearize those relationship and evaluated then at the current estimate:

$$\mathbf{A}_t = \frac{\partial f_t(\mathbf{x}, \mathbf{u})}{\partial \mathbf{x}} \quad \mathbf{B}_t = \frac{\partial f_t(\mathbf{x}, \mathbf{u})}{\partial \mathbf{u}} \quad \mathbf{H}_t = \frac{\partial h_t(\mathbf{x})}{\partial \mathbf{x}}.$$



Extended Kalman Filter

Data: $\mu_{t-1}, \Sigma_{t-1}, \mathbf{u}_t, \mathbf{z}_t$

Result: μ_t, Σ_t

/* Prediction

*/

1 $\bar{\mu}_t = f_t(\mu_{t-1}, \mathbf{u}_t);$

2 $\bar{\Sigma}_t = \mathbf{A}_t \Sigma_{t-1} \mathbf{A}_t^T + \mathbf{R}_t;$

/* Update

*/

3 $\mathbf{K}_t = \bar{\Sigma}_t \mathbf{H}_t^T (\mathbf{H}_t \bar{\Sigma}_t \mathbf{H}_t^T + \mathbf{Q}_t)^{-1};$

4 $\mu_t = \bar{\mu}_t + \mathbf{K}_t (\mathbf{z}_t - h_t(\bar{\mu}_t));$

5 $\Sigma_t = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \bar{\Sigma}_t;$

6 **return** $\mu_t, \Sigma_t;$

Algorithm 4: Extended Kalman Filter



EKF for robot localization

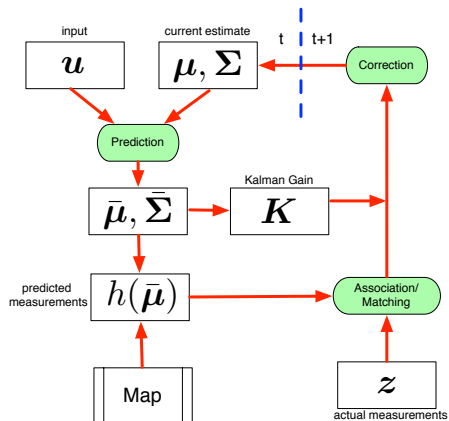


Figure: EKF localization cycle



Pose Tracking with EKF

Motion Model:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_t) + \mathbf{v}_t = \begin{bmatrix} x_{t-1} + l_t \cos(\theta_{t-1}) \\ y_{t-1} + l_t \sin(\theta_{t-1}) \\ \theta_{t-1} + r_t \end{bmatrix} + \mathbf{v}_t.$$

$\mathbf{v}_t$ is a zero mean Gaussian vector with covariance matrix $\mathbf{R}$

$$\mathbf{R} = \begin{bmatrix} 0.2 & 0 & 0 \\ 0 & 0.2 & 0 \\ 0 & 0 & 0.1 \end{bmatrix}$$

The initial state is Gaussian distributed with mean $\mu_0 = [0 \ 0 \ 0]^T$ and covariance

$$\Sigma_0 = \begin{bmatrix} 0.3 & 0 & 0 \\ 0 & 0.1 & 0 \\ 0 & 0 & 0.2 \end{bmatrix}$$



Pose Tracking with EKF

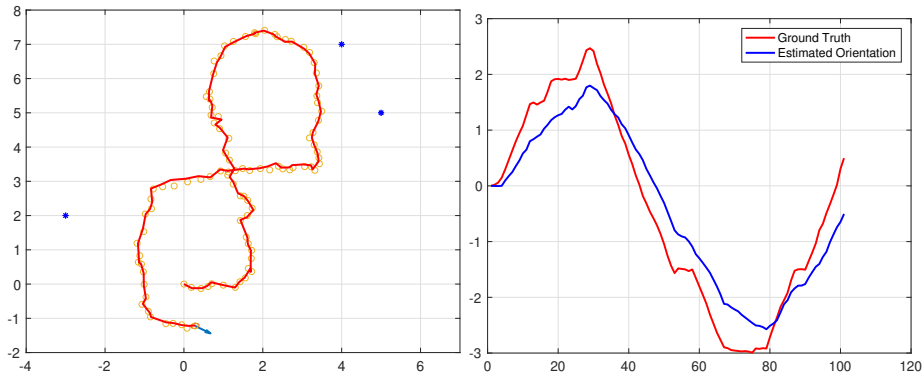
Sensor Model (distance from three distinguishable landmarks)

$$\mathbf{z}_t = h_t(\mathbf{x}_t) + \mathbf{w}_t = \begin{bmatrix} \sqrt{(x_L^1 - x_t)^2 + (y_L^1 - y_t)^2} \\ \sqrt{(x_L^2 - x_t)^2 + (y_L^2 - y_t)^2} \\ \sqrt{(x_L^3 - x_t)^2 + (y_L^3 - y_t)^2} \end{bmatrix} + \mathbf{w}_t$$

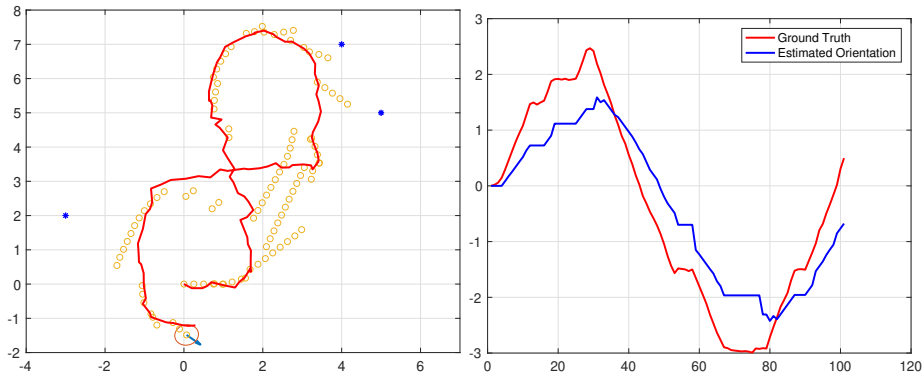
$$\mathbf{Q} = \begin{bmatrix} 0.001 & 0 & 0 \\ 0 & 0.001 & 0 \\ 0 & 0 & 0.001 \end{bmatrix}.$$



Estimation Error – Continuous Measurement

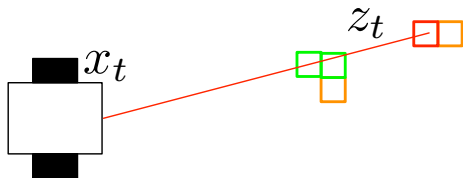


Estimation Error – Sporadic Measurement



Mapping with known poses

Mapping is another estimation problem: assign an occupancy probability to each grid cell.



Assumptions: available sensors model; all cells independent; known robot pose.



Mapping with known poses – derivation

$$p(m_i = 1 | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_n)$$

$$\begin{aligned} p(m_i | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_n) &= \\ &= \frac{p(z_n | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_{n-1}, m_i) p(m_i | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_{n-1})}{p(z_n | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_{n-1})} \\ &= \frac{p(z_n | x_n, m_i) p(m_i | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_{n-1})}{p(z_n | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_{n-1})} \\ &= \frac{p(m_i | x_n, z_n) p(z_n | x_n) p(m_i | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_{n-1})}{p(m_i) p(z_n | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_{n-1})} \\ &= \frac{p(m_i | x_n, z_n) p(z_n | x_n) p(m_i | x_1, x_2, \dots, x_{n-1}, z_1, z_2, \dots, z_{n-1})}{p(m_i) p(z_n | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_{n-1})} \end{aligned}$$



Mapping with known poses – derivation

Next compute the probability of the cell being free:

$$\begin{aligned} p(\neg m_i | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_n) &= \\ &= \frac{p(\neg m_i | x_n, z_n) p(z_n | x_n) p(\neg m_i | x_1, x_2, \dots, x_{n-1}, z_1, z_2, \dots, z_{n-1})}{p(\neg m_i) p(z_n | x_1, x_2, \dots, x_n, z_1, z_2, \dots, z_{n-1})} \end{aligned}$$

Take the ratio between the two estimates and cancel out terms,

$$\frac{p(m_i | x_1, \dots, x_n, z_1, \dots, z_n)}{p(\neg m_i | x_1, \dots, x_n, z_1, \dots, z_n)} = \frac{p(m_i | x_n, z_n) p(\neg m_i) p(m_i | x_1, \dots, x_{n-1}, z_1, \dots, z_{n-1})}{p(m_i) p(\neg m_i | x_n, z_n) p(\neg m_i | x_1, \dots, x_{n-1}, z_1, \dots, z_{n-1})}$$



Mapping with known poses – derivation

$p(m_i) = 1 - p(\neg m_i)$ Introduce odds of a binary random variable:

$$\text{Odds}(x) = \frac{p(x)}{1 - p(x)}.$$

Note it is invertible if $p(x) \neq 1$. Previous ratio becomes:

$$\text{Odds}(p(m_i|x_1, \dots, x_n, z_1, \dots, z_n)) = \frac{\text{Odds}(p(m_i|x_n, z_n)) \text{Odds}(p(m_i|x_1, \dots, x_{n-1}, z_1, \dots, z_{n-1}))}{\text{Odds}(m_i)}$$



Mapping with known poses – derivation

Introduce log for numerical stability:

$$\begin{aligned}\log Odds(p(m_i|x_1, \dots, x_n, z_1, \dots, z_n)) &= \\ &= \log \frac{Odds(p(m_i|x_n, z_n) Odds(p(m_i|x_1, \dots, x_{n-1}, z_1, \dots, z_{n-1})))}{Odds(m_i)} \\ &= \log Odds(p(m_i|x_n, z_n)) + \log Odds(p(m_i|x_1, \dots, x_{n-1}, z_1, \dots, z_{n-1})) - \log Odds(m_i)\end{aligned}$$



Mapping with known poses

Data: $x_n = (x, y, \vartheta)$, z_n , l_{n-1} , m_i

Result: l_n

```
1 if  $m_i$  in perceptual range of  $x_n, z_n$  then  
2   |  $l_n^i \leftarrow \log \text{Odds}(m_i | x_n, z_n) + l_{n-1}^i - \log \text{Odds}(m_i);$   
3 else  
4   |  $l_n^i \leftarrow l_{n-1}^i;$   
5 return  $l_n;$ 
```

Algorithm 5: Mapping with known poses

